



دانشکده مهندسی برق

مدارهای منطقی و سیستم‌های دیجیتال

آزمایش 3 - آشنایی با Verilog
FPGA و تراشه Zynq

آزمایش ۳ - آشنایی با Verilog، FPGA و تراشه Zynq

در کلاس با مدارات ترکیبی و زبان توصیف سخت افزار Verilog آشنا شدید. در این آزمایش به کمک زبان Verilog، به توصیف چند مدار ترکیبی می پردازیم. در ادامه با FPGA، نرم افزار Vivado و تراشه Zynq آشنا می شویم. شما در این آزمایش ضمن یادگیری نحوه کار با زینک، مدارات ترکیبی داده شده را توصیف کرده و در بخش FPGA زینک پیاده سازی و تست می کنید.

قطعات و تجهیزات مورد نیاز

- برد Zynq (AXPZ7010)
- یک عدد کابل USB و سیم پاور برد
- نرم افزار Vivado 2019.1

۱- پیش مطالعه (اختیاری)

برای آشنایی با مفاهیمی چون FPGA، HDL (Verilog) و تراشه Zynq به بخش اطلاعات تکمیلی آزمایش سوم در انتهای این فایل مراجعه کنید.

۲- پیش گزارش

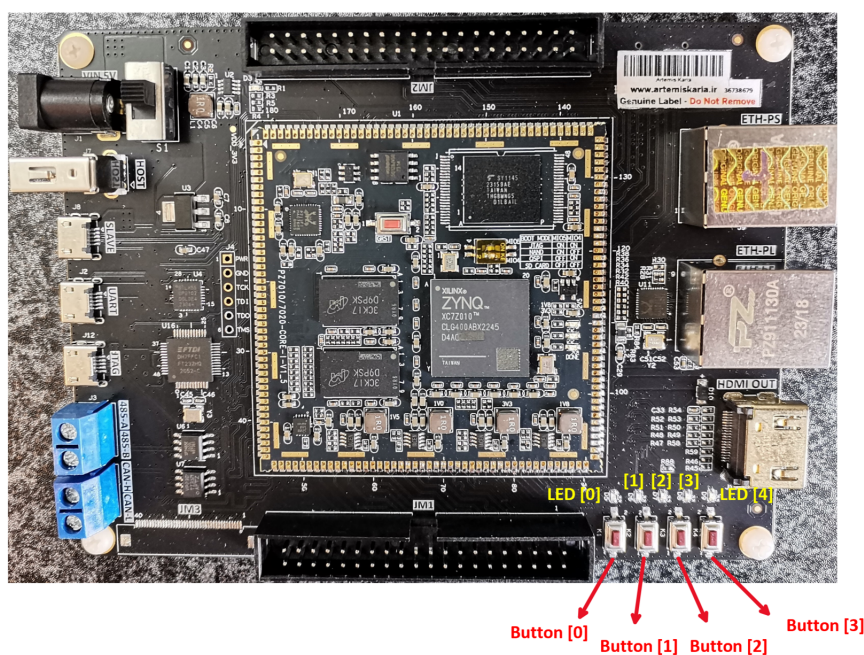
پیش از شروع، برای آمادگی قبل از آزمایش، حتماً ویدیوهای هفته اول تا سوم این جدول برنامه را مشاهده کرده باشید.

۳- دستور کار

۱-۳ آشنایی با برد

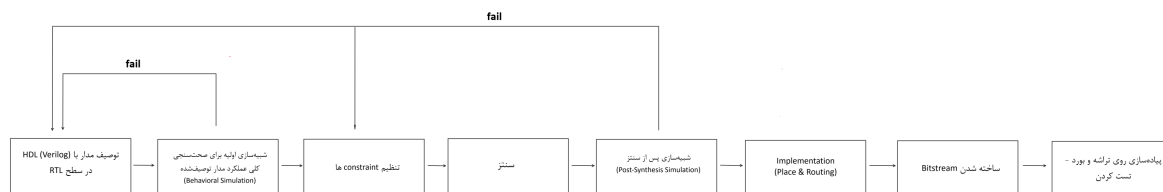
بردی که در آزمایشگاه با آن کار می‌کنید شامل تراشه Zynq 7010 می‌باشد. برای دیدن نتیجه و تست مدارات ساده ساخته شده و برقراری ارتباط با دنیای خارج، لازم است بتوانیم ورودی و خروجی‌های مدارها را توسط IO های برد مدیریت و مشاهده کنیم؛ در واقع ورودی‌های مدار را به یک سری کلید متصل کرده و خروجی‌ها را به LED متصل می‌کنیم تا عملکرد مدار را پس از پیاده‌سازی روی تراشه تست کنیم. برای این منظور برد FPGA مورد استفاده در آزمایشگاه (که دارای یک چیپ مدل Zynq 7010 ساخته شده شرکت Xilinx است). دارای چهار کلید ورودی به صورت ساده (push button) و پنج LED تک می‌باشد که همگی به پین‌های طرف (FPGA) PL تراشه متصل شده‌اند.

⚠ توجه کنید که LED ها Active High و کلیدها Active Low هستند. برای اینکه در نوشتن کد و کار کردن با ورودی‌های Active Low دچار سردرگمی نشوید، نقیض آن‌ها را به یک wire متصل کنید و با متغیر جدید به نوشتن منطق مورد نظرتان ادامه دهید.



شکل ۱-۳: برد Zynq آزمایشگاه مدار منطقی (AXPZ7010)، دارای تراشه Zynq 7010

هرکدام از کلیدها و LEDها در PCB بورد به یکی از پین‌های FPGA متصل شده‌اند که در ادامه به طور دقیق به این موضوع اشاره خواهیم کرد.



شکل ۳-۲: مراحل FPGA design Flow

اولین مرحله در کار با FPGAها توصیف مدار مورد نظر با زبان وریلاگ است.

⚠ در کلیه بخش‌های این آزمایش لازم است فقط از گیت‌های درونی و عبارت‌های Continuous Assignment استفاده گردد.

نام ماژول و ورودی خروجی‌های ماژول شما می‌تواند هر اسم دلخواهی باشد، اما در این آزمایش برای سهولت اتصال ورودی خروجی‌های ماژول به کلیدها و LEDهای بورد (که در ادامه توضیح خواهیم داد)، از فرمت کلی زیر استفاده کنید (این فرمت برای Top Module هر قسمت است که بناست به IOهای بورد وصل شود).

```

`timescale 1ns / 1ps

module top_module(
    input wire [3:0] button,
    output wire [4:0] LED);

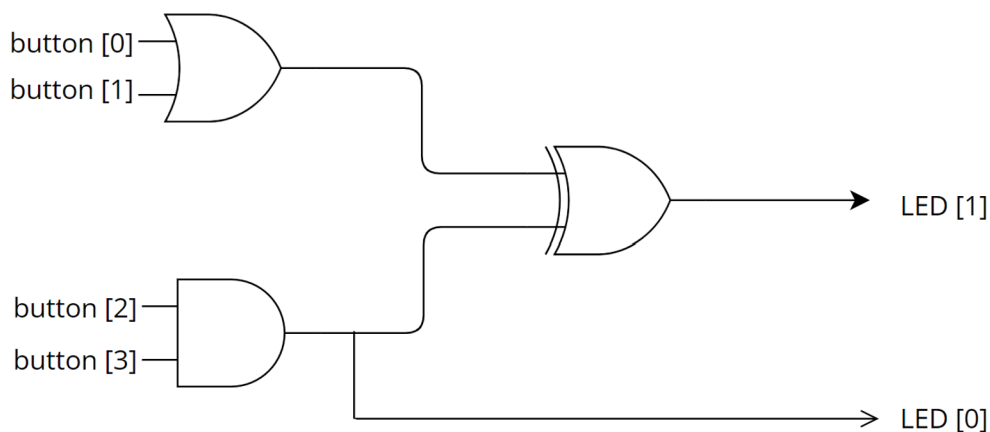
    // Your Code Here

endmodule
  
```

این آزمایش دارای پنج بخش کلی است؛ در بخش اول آزمایش توضیح انجام مراحل با جزئیات کامل داده شد و هدف این است که شما برای اولین تجربه کار با Vivado مشکل نداشته باشید و منبعی برای مراجعه دوباره شما به جهت انجام مراحل وجود داشته باشد. پس از آن در بخش‌ها و آزمایش‌های بعدی، صرفاً تسک مربوط به هر بخش توضیح داده شده و برای انجام مراحل به صورت پیش فرض به توضیحات بخش اول دستور کار این آزمایش مراجعه کنید.

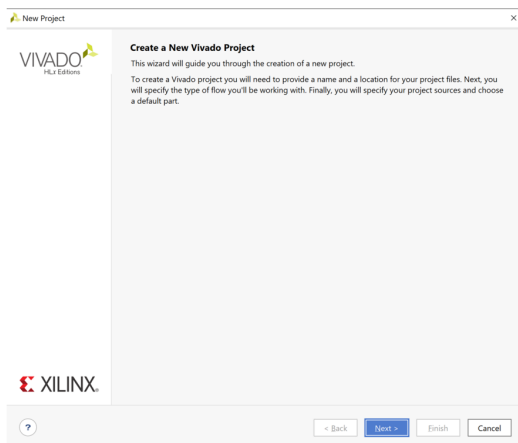
۳-۲- بخش اول (زمان تقریبی: ۳۰ دقیقه)

مدار زیر را به زبان وریلاگ و فقط با استفاده از عبارات Continuous Assignment توصیف کنید.

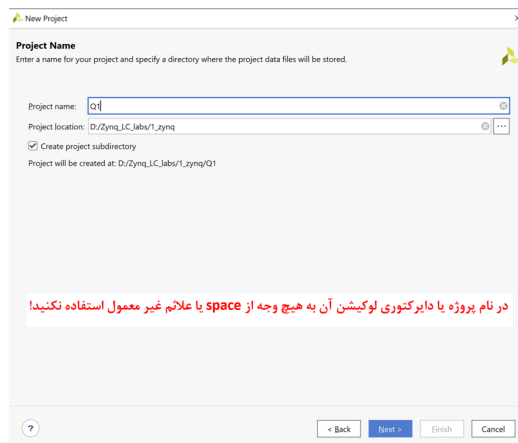


توصیه می‌شود از **VSCode** به عنوان ادیتور برای نوشتن کد وریلاگ استفاده کنید. سپس وارد نرم‌افزار Vivado شده و مراحل زیر را طی کنید.

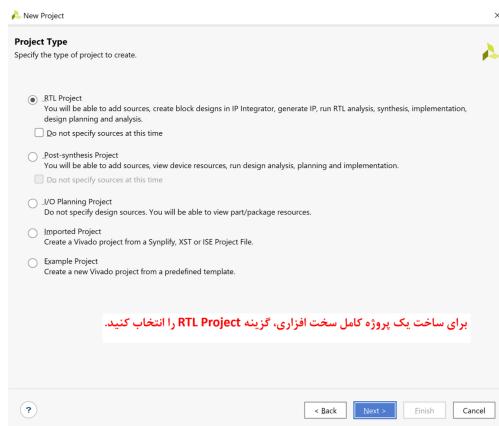
1.



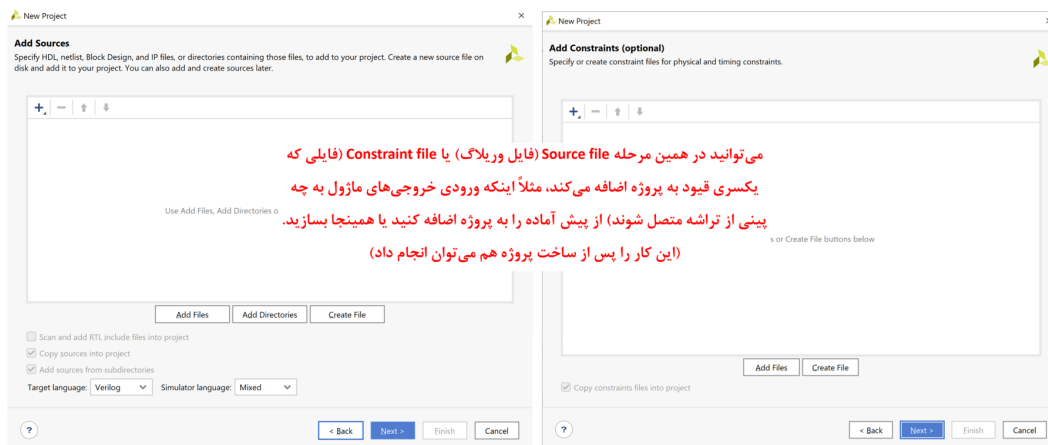
2.



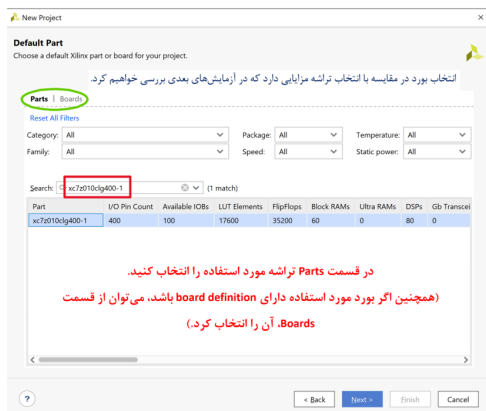
3.



4.



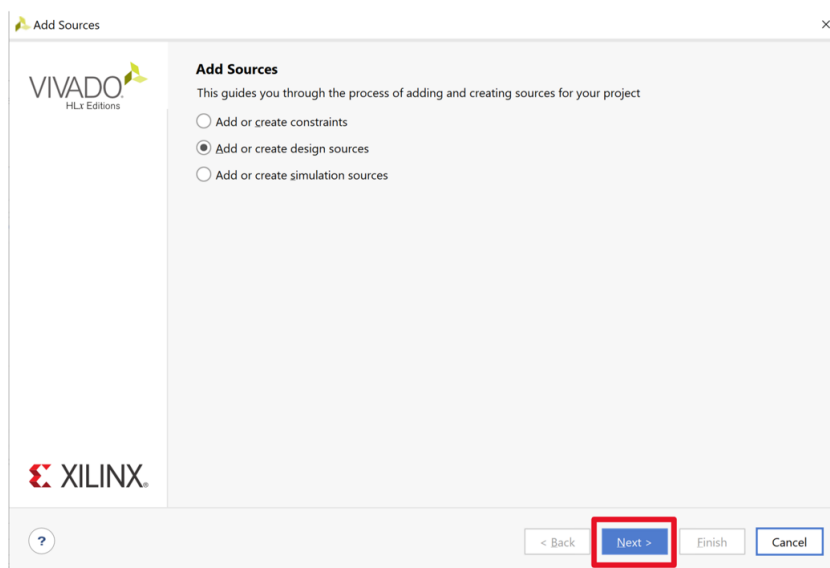
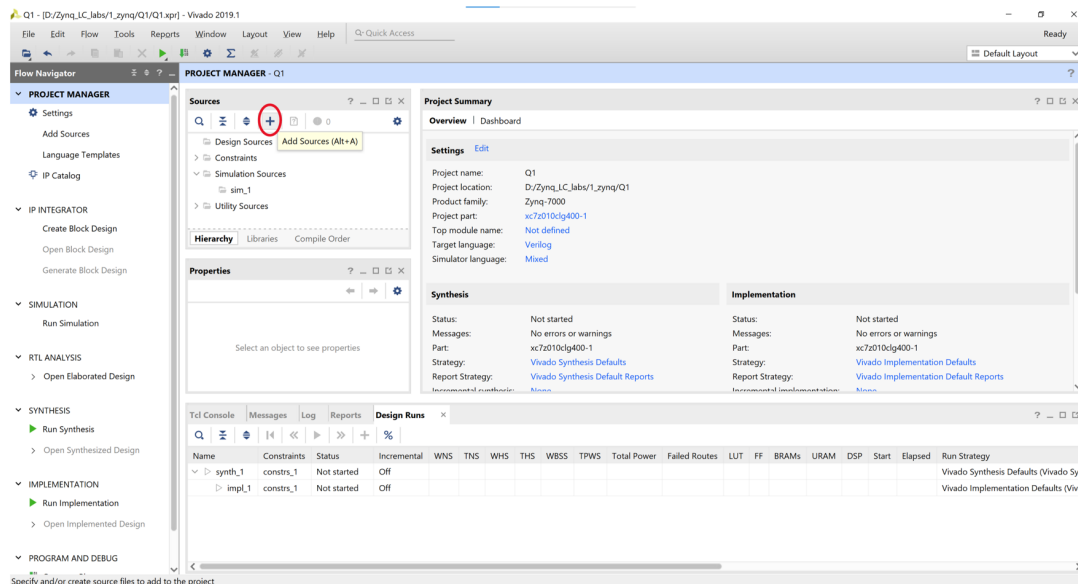
5.

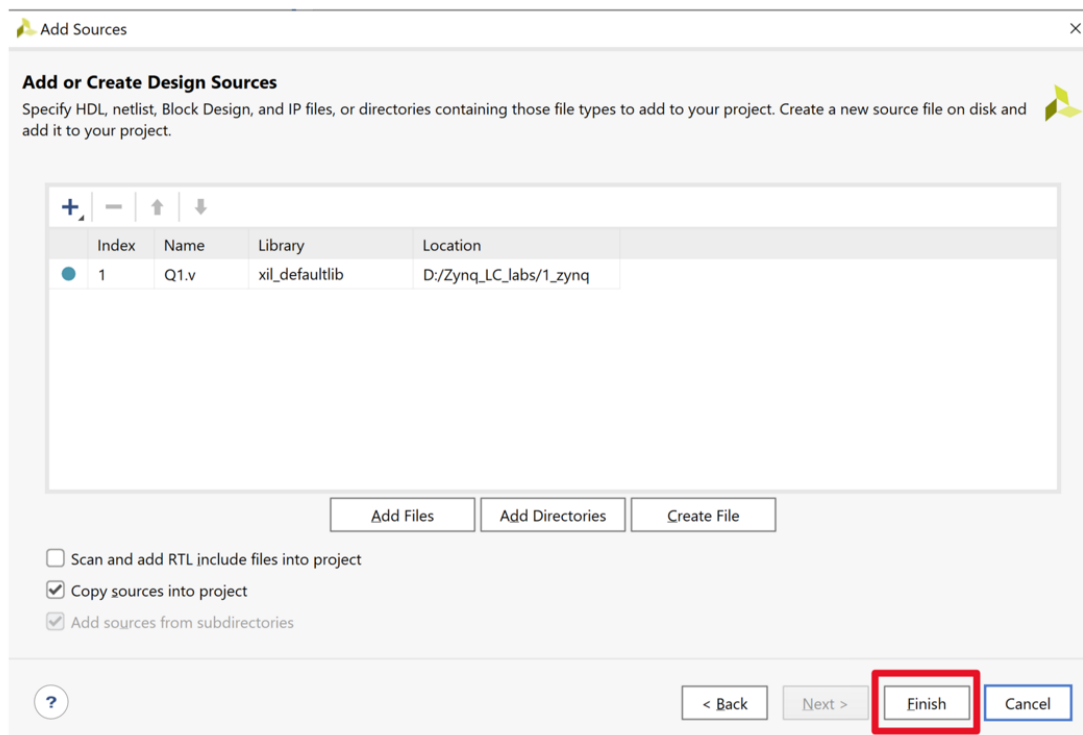
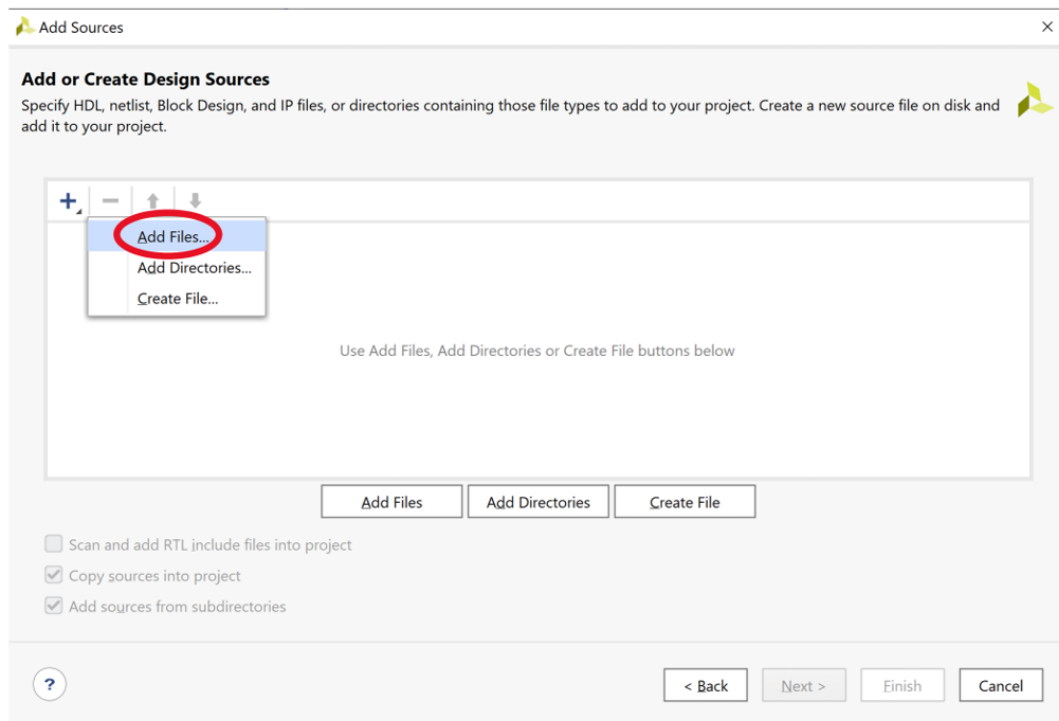


6.



حال مانند تصاویر زیر فایل وریلاگ نوشته شده را به عنوان Source به پروژه اضافه کنید.





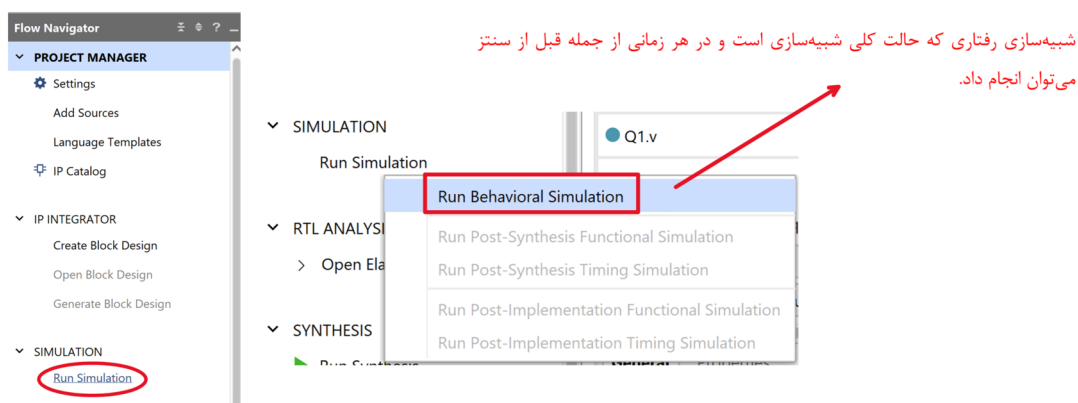
ادیتور Vivado به صورت آنلاین ایرادات سیستمی کد شما را بررسی می‌کند و علامت سبز رنگ نشان‌دهنده صحت کد از این جهت می‌باشد:



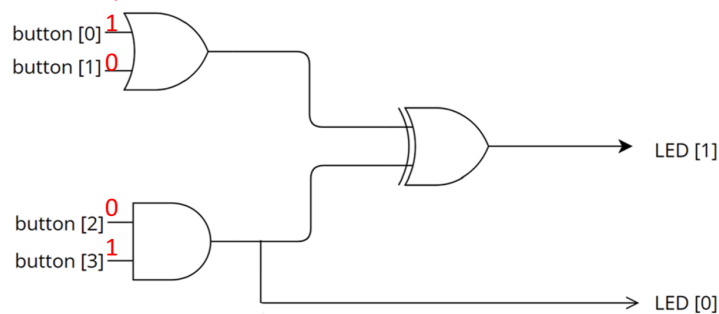
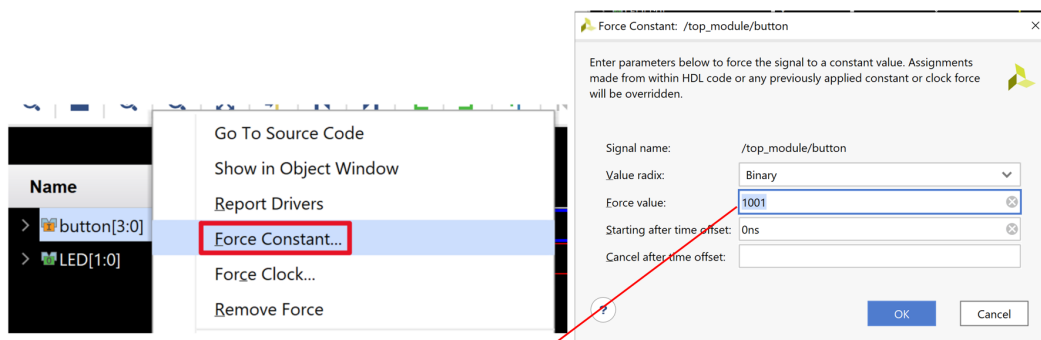
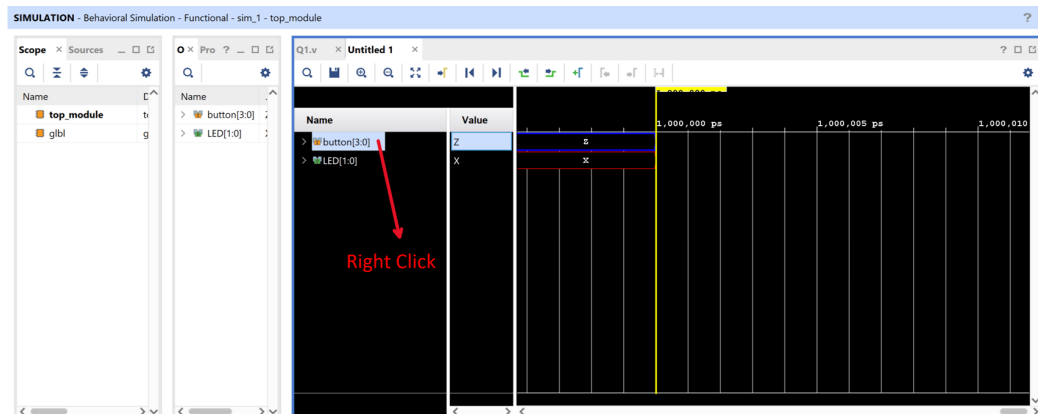
در ادامه کد خود را شبیه‌سازی کنید تا از صحت عملکرد مدار حاصل از آن مطمئن شوید.

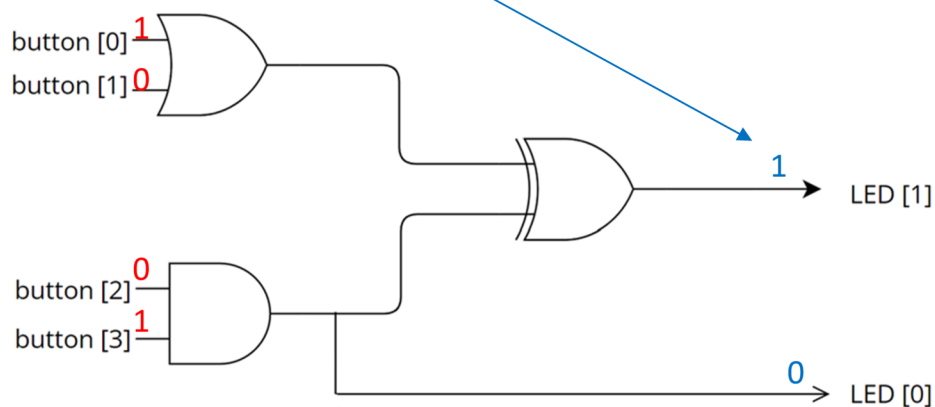
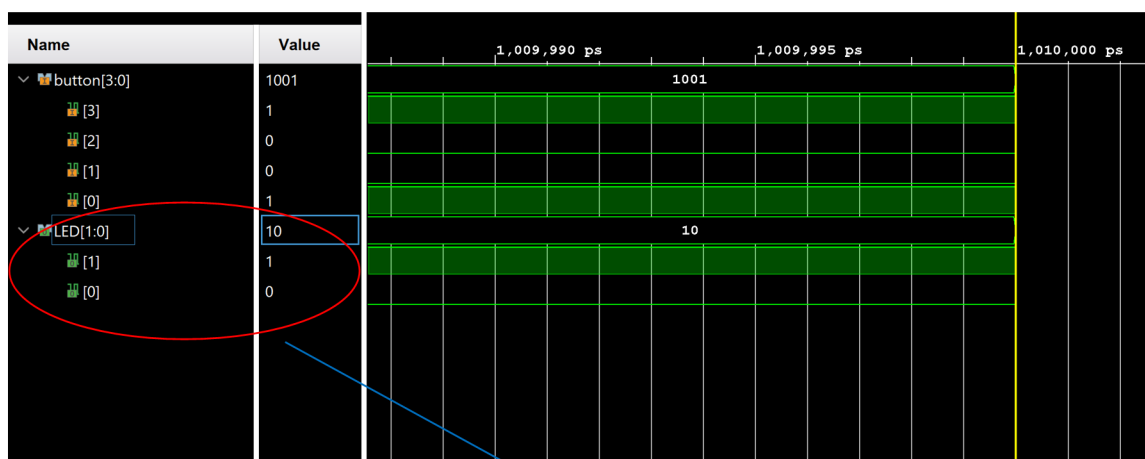
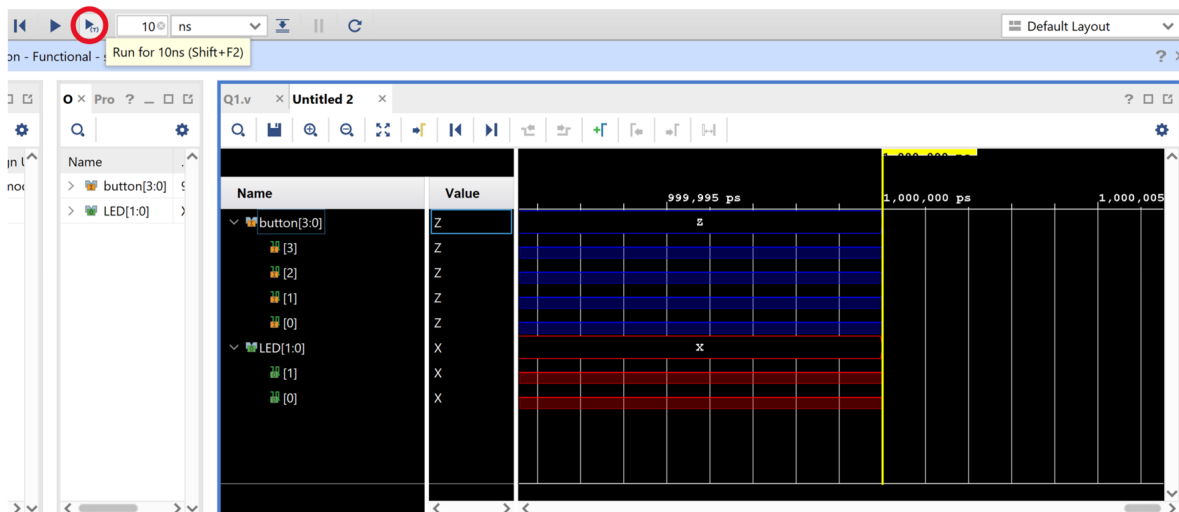
۳-۲-۲- مراحل شبیه‌سازی در Vivado

(این مراحل برای شبیه‌سازی بدون testbench است که در شبیه‌سازی‌های محدود و مدارات ساده کاربرد دارد)



نحوه دادن مقدار به ورودی‌ها در شبیه‌سازی:



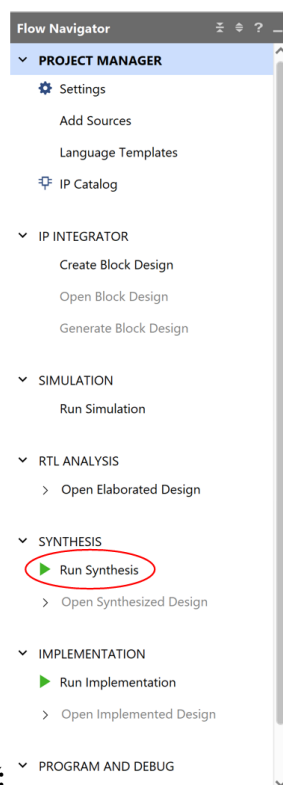


برای اطمینان از صحت عملکردی کد، چند مقدار دیگر را به صورت دستی امتحان کنید. سپس برای صحت سنجی مدار آن ها را به دستیاران آزمایشگاه ارائه دهید..

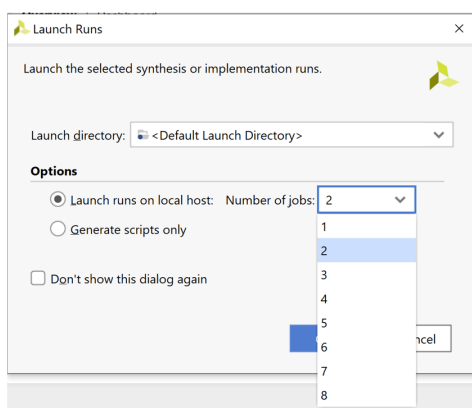
۳-۲-۳- مراحل پیاده سازی روی برد

الف) سنتز: در مرحله اول باید کد وریلاگ نوشته شده توسط شما به مداری شامل گیت ها و عناصر منطقی (LUT ها و ...) تبدیل شود. هم چنین فرآیندهای بهینه سازی Logic به صورت خودکار انجام می گیرد. به این فرآیند سنتز مدار گفته می شود که در این مثال توسط Synthesis Tool نرم افزار Vivado انجام می شود. در این مرحله اهمیتی ندارد که از چه چپیی استفاده می کنید یا ورودی خروجی های مدار شما به چه پین هایی از تراشه متصل خواهند شد، چرا که صرفا قرار است شماتیکی از مدار شما طراحی شود؛ پیاده سازی آن روی تراشه برای مراحل بعدی است.

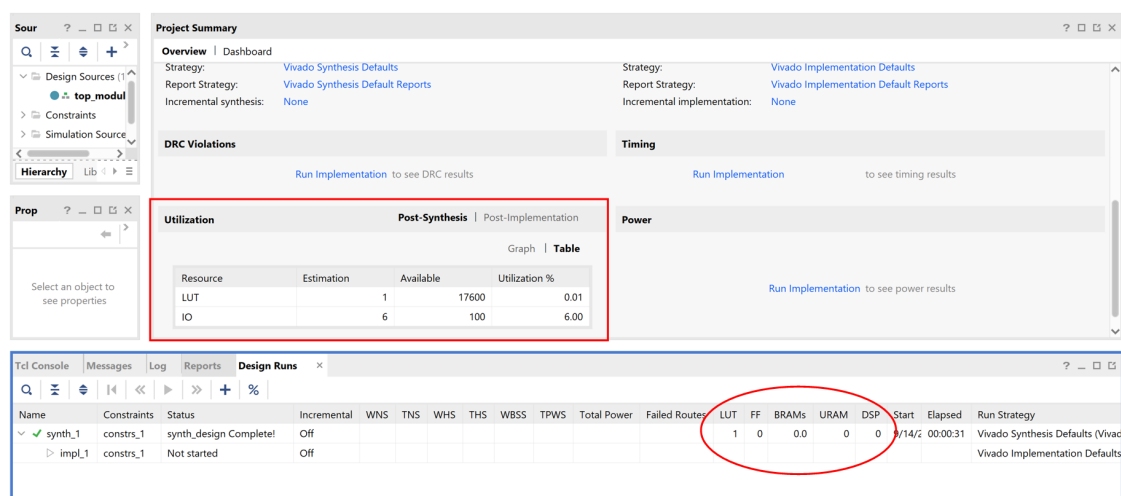
برای سنتز شدن کد در Vivado بر روی گزینه زیر در بخش SYNTHESIS از پنجره Flow Navigator کلیک کنید.



سپس پنجره‌ای باز شده و از شما می‌خواهد مشخص کنید فرآیند سنتز بر روی چند هسته از پردازنده کامپیوتر شما اجرا شود. این انتخاب از این جهت مهم است که در پروژه‌های بزرگتر که شامل چندین IP Core (در ساده‌ترین حالت یک ماژول از پیش آماده وریلاگ تصورش کنید) و ماژول سخت‌افزاری هستند، Vivado می‌تواند برای سنتز هر کدام از آن‌ها سناریوی جداگانه‌ای در نظر گرفته و آن‌ها را به صورت موازی روی هسته‌های مختلف کامپیوتر انجام دهد تا سرعت سنتز بالاتر برود. در این بخش چون تنها یک ماژول وریلاگ داریم، تفاوتی ندارد این تنظیم را روی چند هسته قرار دهیم (توجه کنید که هیچ‌گاه از تمام هسته‌های پردازنده‌تان برای سنتز استفاده نکنید؛ چرا که سرعت کلی کار کامپیوتر در تعامل با سایر برنامه‌هایی که باز هستند، کم می‌شود).



پس از سنتز، میزان منابع مصرفی استفاده شده توسط Synthesis tool برای ساخت مدار را می‌توانید از پنجره Project Summary یا تب Design Runs مشاهده کنید. در خصوص تعداد منابع مصرف شده بحث کنید و آن را در گزارش آزمایش قید کنید.



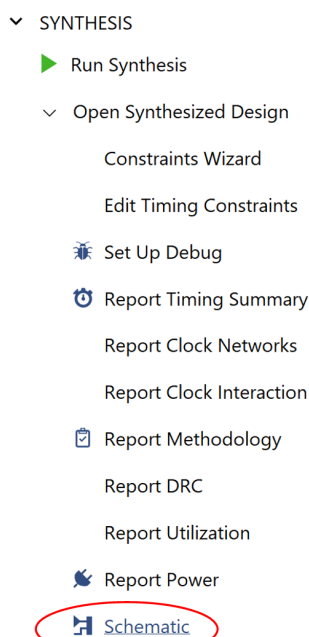
Project Summary - Utilization

Resource	Estimation	Available	Utilization %
LUT	1	17600	0.01
IO	6	100	6.00

Design Runs

Name	Constraints	Status	Incremental	WNS	TNS	WHS	THS	WBSS	TPWS	Total Power	Failed Routes	LUT	FF	BRAMS	URAM	DSP	Start	Elapsed	Run Strategy
synth_1	constrs_1	synth_design Complete!	Off									1	0	0.0	0	0	9/14/2	00:00:31	Vivado Synthesis Defaults (Vivado)
impl_1	constrs_1	Not started	Off																Vivado Implementation Defaults

حال با کلیک بر روی Schematic در بخش SYNTHESIS، شماتیک مدار خود را پس از سنتز مشاهده کرده و با شکل مدار رسم شده مقایسه کنید (به این تفاوت و نوع عناصر استفاده شده در مدار سنتز شده توسط Synthesis Tool دقت کنید و در گزارش بنویسید).



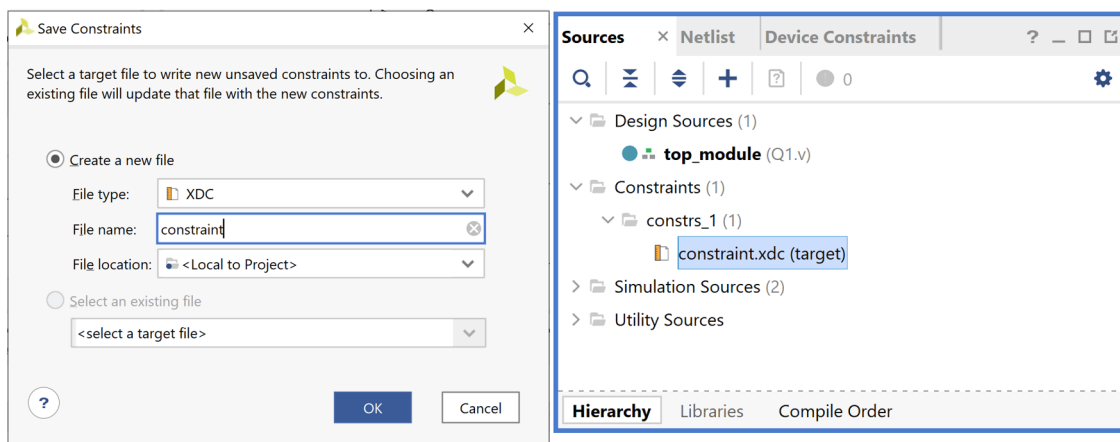
این پورت‌هایی (ورودی خروجی‌هایی) که تا به حال در کد وریلاگ با آن‌ها کار کردید، پورت‌های انتزاعی هستند و برای پیاده‌سازی مدار بر روی FPGA باید آن‌ها را به کلیدها، LEDها و به طور کلی Interface‌های روی برد متصل کنید. لذا هر پورت مدار خود را به پینی از تراشه متصل می‌کنید که از نظر سخت‌افزاری روی PCB برد به یکی از این Interface‌ها متصل است.

در جدول زیر مشخص شده که هر کدام از این کلیدها و LEDها در PCB برد به کدام یک از پین‌های FPGA وصل شده‌اند. از این جدول شما باید برای تنظیم Constraint در Vivado استفاده کنید؛ یعنی مشخص کنید که هر ورودی و خروجی ماژول وریلاگ (مدار) شما به چه پینی از FPGA متصل شود.

Name	Package Pin	Description - I/O Std
Button[0]	T10	BANK34 FIX 3.3V
Button[1]	T11	
Button[2]	T15	
Button[3]	T14	
LED[0]	V20	LVCMOS33
LED[1]	W19	
LED[2]	W18	
LED[3]	V18	
LED[4]	V17	

جدول : 1-3 اطلاعات پین های متصل به کلید ها و LED های برد

حال در بخش I/O Ports ورودی خروجی های مدار سنتز شده را مطابق جدول به پین های مناسبی از تراشه (که از نظر سخت افزاری و PCB برد، به کلیدها و LED های برد متصل اند) Assign کنید. سپس با ذخیره کردن آن، فایل را با پسوند و فرمت XDC تولید می شود که اصطلاحاً به آن Constraint File می گوئیم.



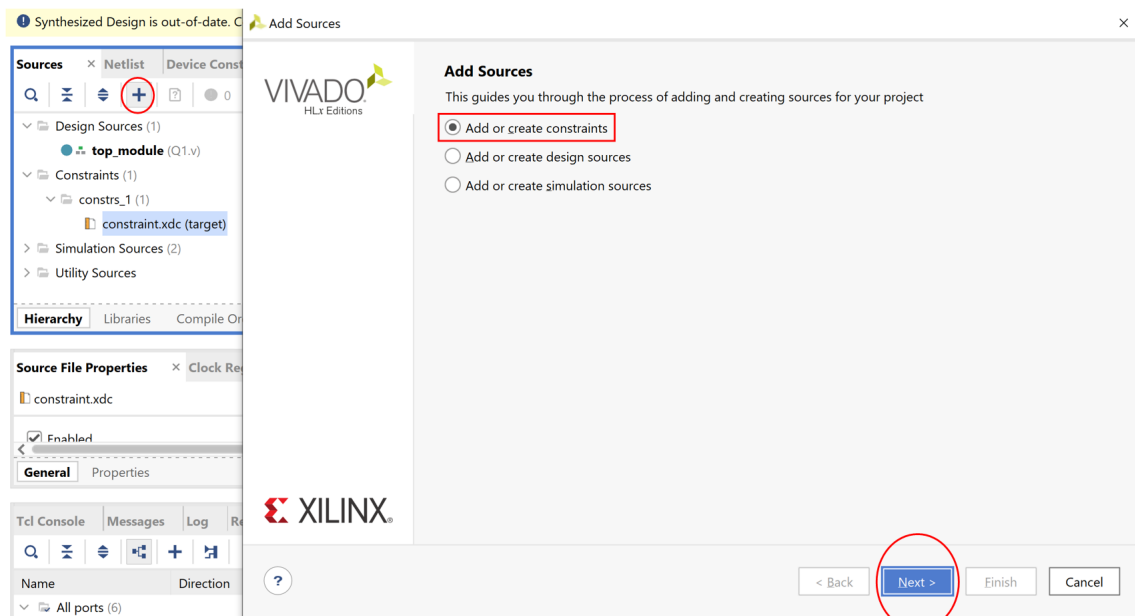


```

1 set_property PACKAGE_PIN T10 [get_ports {button[0]}}
2 set_property PACKAGE_PIN T11 [get_ports {button[1]}}
3 set_property PACKAGE_PIN T15 [get_ports {button[2]}}
4 set_property PACKAGE_PIN T14 [get_ports {button[3]}}
5 set_property PACKAGE_PIN V20 [get_ports {LED[0]}}
6 set_property PACKAGE_PIN W19 [get_ports {LED[1]}}
7 set_property IOSTANDARD LVCMOS33 [get_ports {button[2]}}
8 set_property IOSTANDARD LVCMOS33 [get_ports {button[1]}}
9 set_property IOSTANDARD LVCMOS33 [get_ports {button[3]}}
10 set_property IOSTANDARD LVCMOS33 [get_ports {button[0]}}
11 set_property IOSTANDARD LVCMOS33 [get_ports {LED[1]}}
12 set_property IOSTANDARD LVCMOS33 [get_ports {LED[0]}}
13
14

```

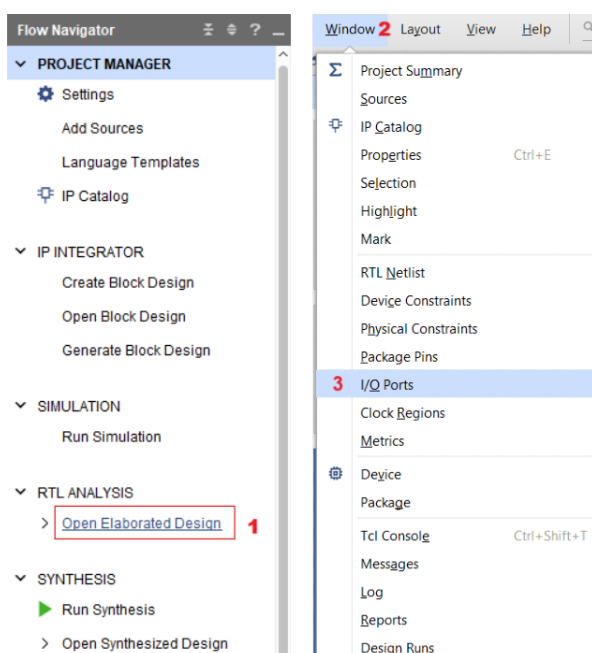
در Constraint File تمام محدودیت‌ها و قیودی که برای ساخته شدن مدار و قرارگیری آن درون FPGA اعمال می‌کنیم، با فرمتی که برای Vivado قابل فهم باشد نوشته شده است؛ یکی از این قیود همان اتصال پورت‌های مدار به پین‌های مناسب تراشه است (البته Constraint File فقط برای این کار نیست).



برای راحتی کار شما، Constraint File مربوط به کلیدها و LEDهای روی برد در قالب یک فایل به فرمت XDC در این لینک قابل دسترسی است. این فایل را به عنوان Constraint در تمام بخش‌های این آزمایش اضافه کنید.

⚠ توجه کنید که ورودی خروجی‌های Top Module شما حتما باید به شکل این **کد** باشد (حتی اگر در بخشی از آزمایش از برخی از کلیدها یا LEDها استفاده نمی‌کنید) تا فایل Constraint به درستی اعمال شود.

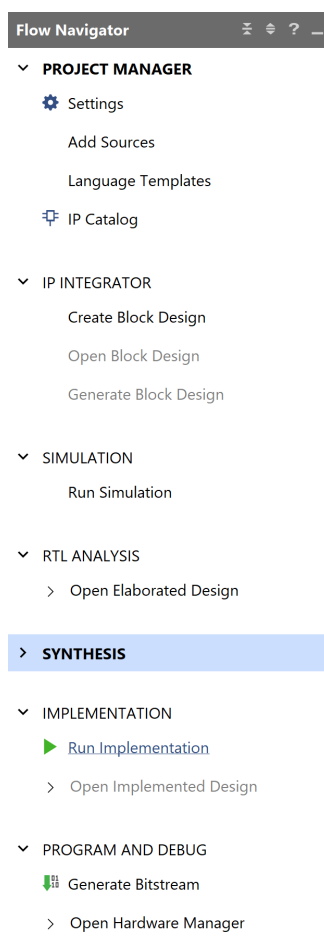
تنظیم اتصال ورودی خروجی‌ها به پین مناسب به صورت گرافیکی نیز امکان پذیر است. ابتدا بر روی گزینه open elaborated design کلیک کرده و منتظر بمانید تا دیزاین باز شود. سپس از منوی Window گزینه I/O Ports را انتخاب کنید.



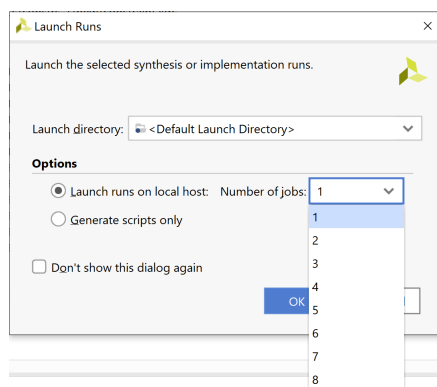
حال ستون Package Pin با پین‌های مربوطه پر کرده و در نهایت I/O Std را بر روی LVCMOS33 تنظیم کنید. برای ذخیره سازی این constraint ها دکمه های Ctrl+S را فشرده و آن را تحت فایل XDC ذخیره کنید.

Name	Direction	Neg Diff Pair	Package Pin	Fixed	Bank	I/O Std	Vcco	Vref	Drive Strength	Slew Type	Pull Type	Off-Chip
button (4)	IN					LVCMOS33*	3.300				NONE	NONE
button[3]	IN					LVCMOS33*	3.300				NONE	NONE
button[2]	IN					LVCMOS33*	3.300				NONE	NONE
button[1]	IN					LVCMOS33*	3.300				NONE	NONE
button[0]	IN					LVCMOS33*	3.300				NONE	NONE
LED (2)	OUT					LVCMOS33*	3.300	12			NONE	FP_VTT_1
LED[1]	OUT					LVCMOS33*	3.300	12			NONE	FP_VTT_1
LED[0]	OUT					LVCMOS33*	3.300	12			NONE	FP_VTT_1
Scalar ports (0)												

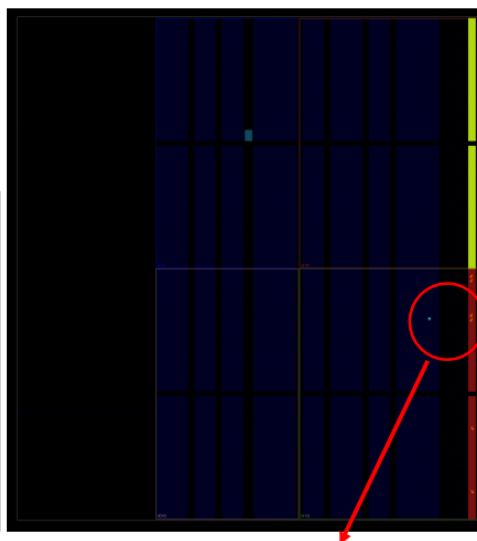
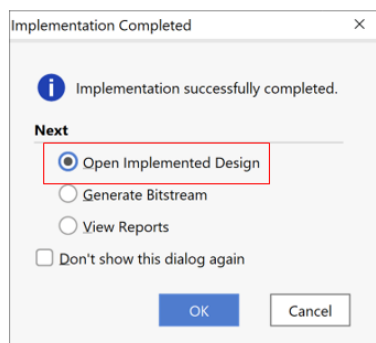
ب) **Implementation**: در این مرحله عملاً مدار سنتز شده قرار است بر روی FPGA قرار گیرد. در ساده‌ترین حالت همانند قرار دادن گیت‌ها بر روی برد و سپس سیم‌کشی بین آنهاست. پس مهم است که از چه تراشه‌ای با چه امکانات و میزان منابع مصرفی می‌کنیم. الگوریتم‌های به کار رفته برای implement کردن هم نسبتاً پیچیده و از موضوع درس خارج است، ولی خوشبختانه توسط کامپوتر و Vivado انجام و بسیار ساده‌تر از استفاده از عناصر فیزیکی است.

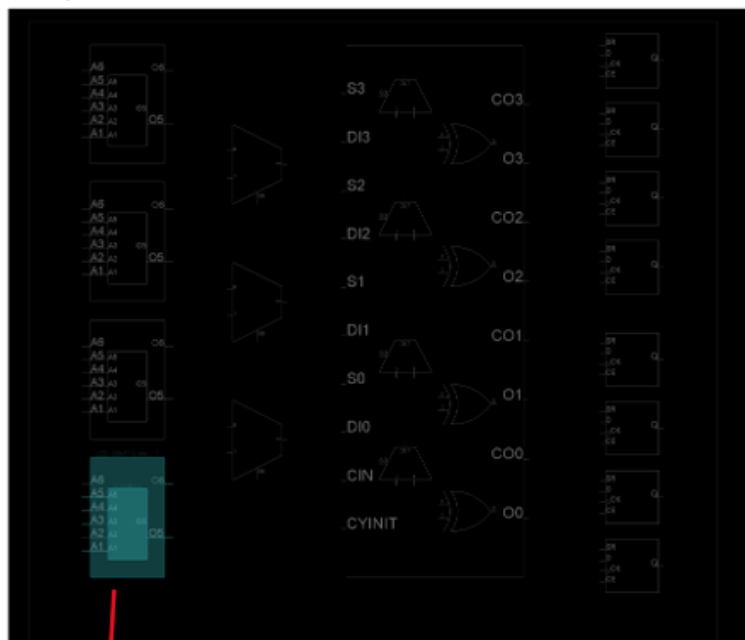


فرایند Implementation به جهت الزامات سیم‌کشی یا برقراری ارتباط بین بخش‌های مختلف، یک فرایند ترتیبی است؛ لذا تعداد هسته پردازنده که به آن اختصاص می‌دهید اهمیتی ندارد و با همان ۱ یا ۲ هسته به ماکزیمم سرعت انجام این فرایند می‌رسیم. (بعضاً اگر تعداد هسته‌های زیادی را درگیر کنید به جهت درگیری بیجای آنها سرعت کلی کامپوتر شما و به تبع آن فرایند implementation کندتر هم می‌شود).



پس از اتمام implementation با زدن گزینه زیر می‌توانید مشاهده کنید که مدار شما به چه صورت و در چه بخشی از تراشه (FPGA) پیاده شده است:



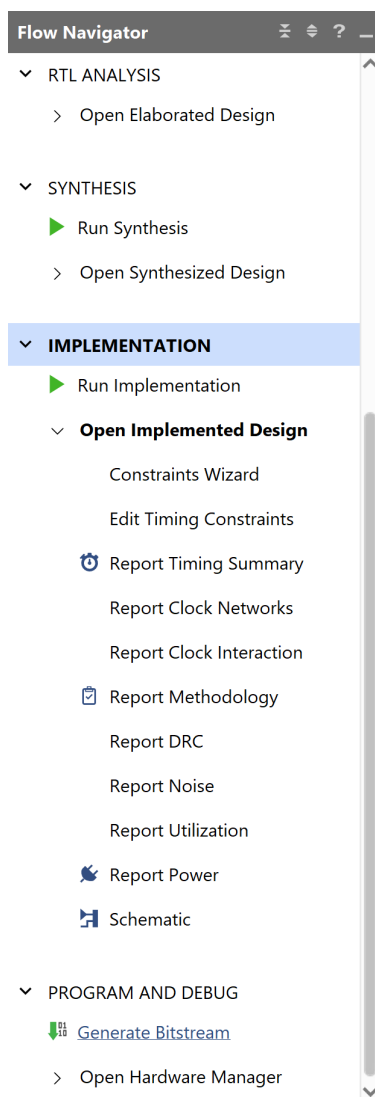


LUT (Look Up Table) مورد استفاده برای پیاده‌سازی گیت‌های مدار

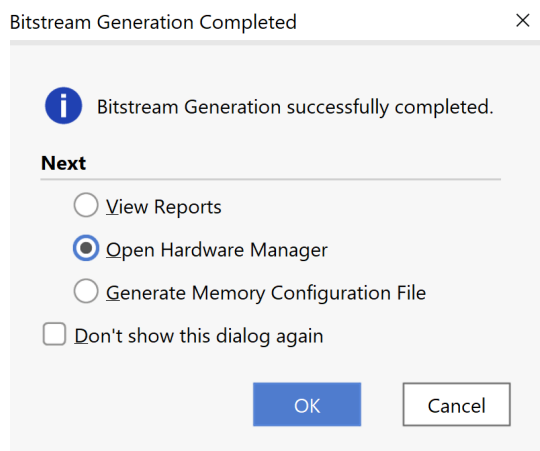
پین‌هایی از تراشه که به عنوان IO مورد استفاده واقع شده اند و ورودی خروجی‌های مدار به آنها وصل شده اند. (پین T14 و T15 که طبق جدول پین‌ها، از نظر سخت‌افزاری در PCB مورد به Button[2] و Button[3] متصل اند)



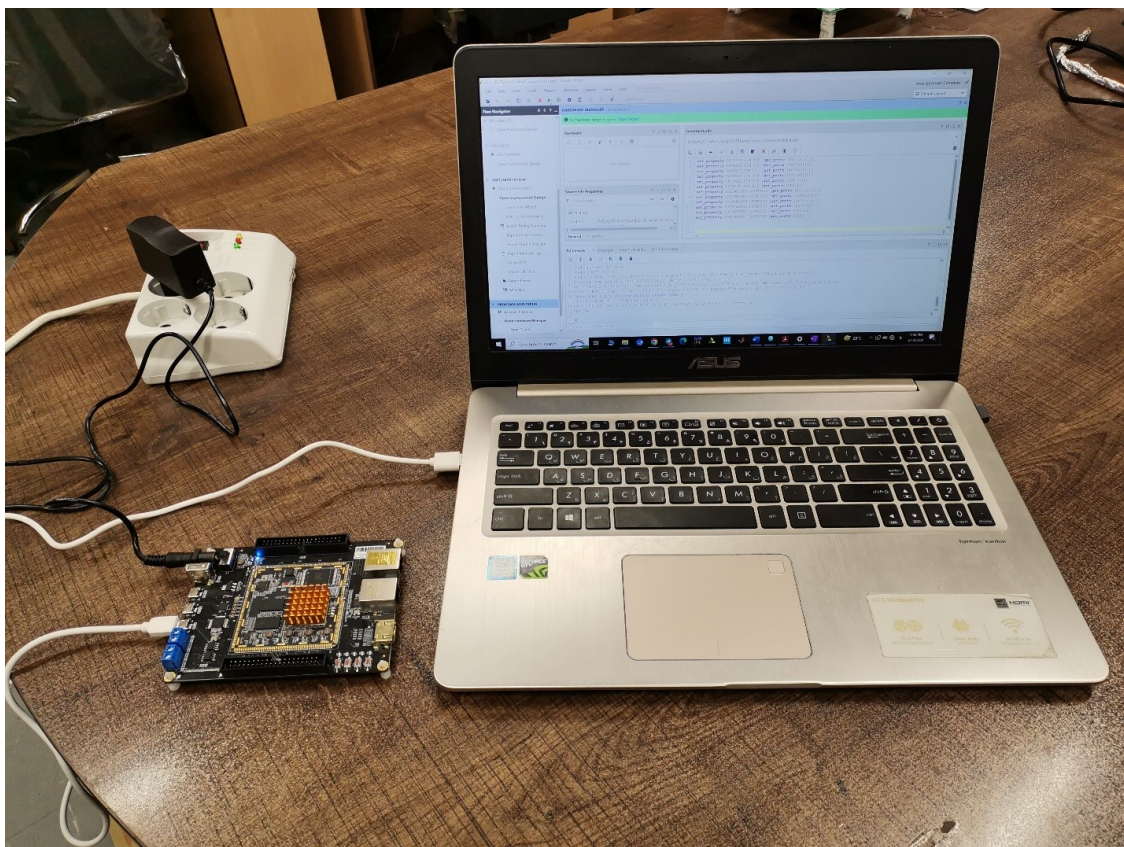
ج) ساخته شدن **Bitstream**: در مرحله بعد باید اطلاعات مدار **implement** شده به قالبی در بیاید که پروگرامر تراشه آن را بشناسد و آن را برنامه‌ریزی کند. به این قالب مشخص **File Bit** و به فایل حاصل از این مرحله **Bitstream** گفته می‌شود که با فرمت **.bit** ساخته و ذخیره می‌شود. در ساده‌ترین حالت تصور کنید این فایل شامل تعدادی ۰ و ۱ است که پروگرامر بر اساس این استریم از بیت‌ها متوجه می‌شود کدام سوئیچ بین کدام دو گیت درونی **FPGA** را به هم متصل بکند.



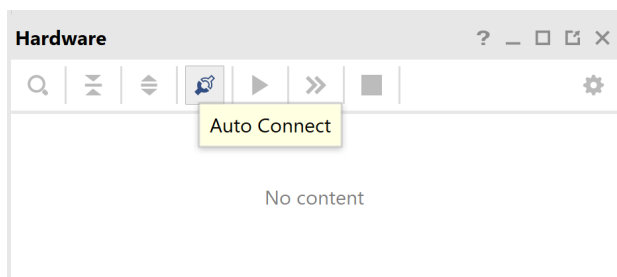
د) پروگرام کردن تراشه و بورد: پس از ساخته شدن **Bitstream** با انتخاب گزینه زیر وارد **Hardware Manager** شوید: **Hardware Manager** را از قسمت **PROGRAM AND DEBUG** در **Flow Navigator** نیز می‌توان باز کرد).



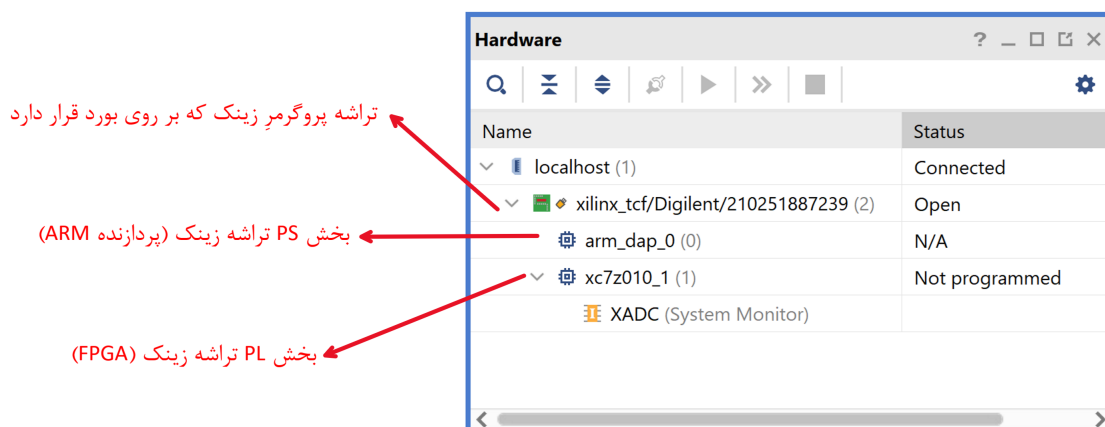
حال برد را از طریق پورت JTAG به کامپیوتر خود متصل کرده و پاور برد را نیز روشن کنید:



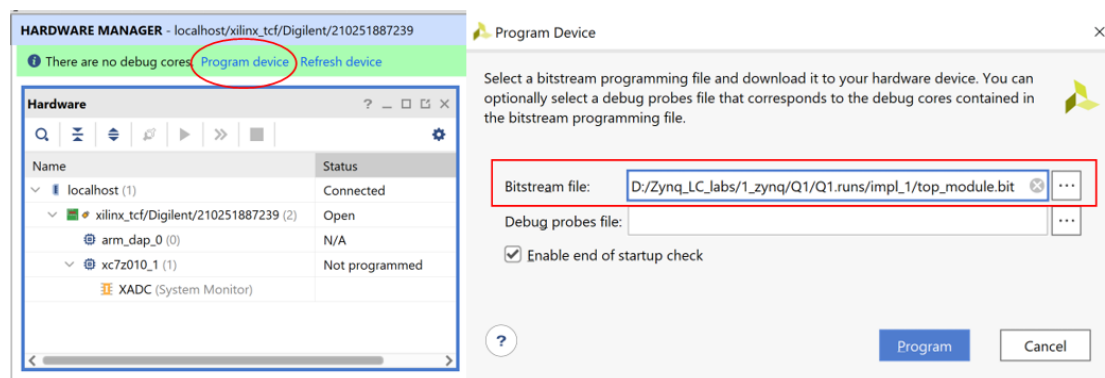
سپس Auto Connect را بزنید تا به طور خودکار تراشه را بشناسد:



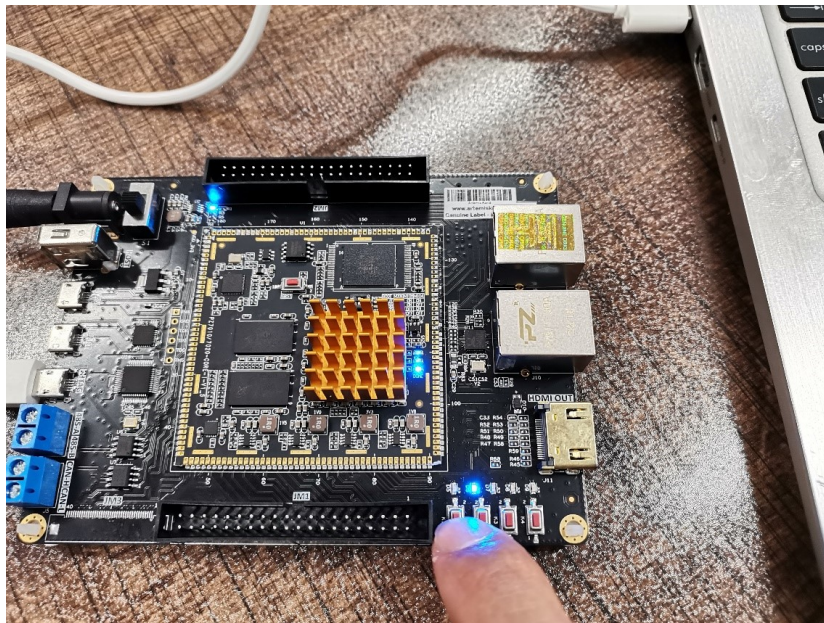
Vivado (کامپیوتر شما) با اتصال به پورت JTAG برد و از طریق آن، ابتدا IC پروگرامر تراشه زینک را می‌شناسد که از لحاظ سخت‌افزاری به پورت JTAG متصل است؛ سپس به واسطه پروگرامر، بخش PS و PL زینک را پیدا می‌کند:



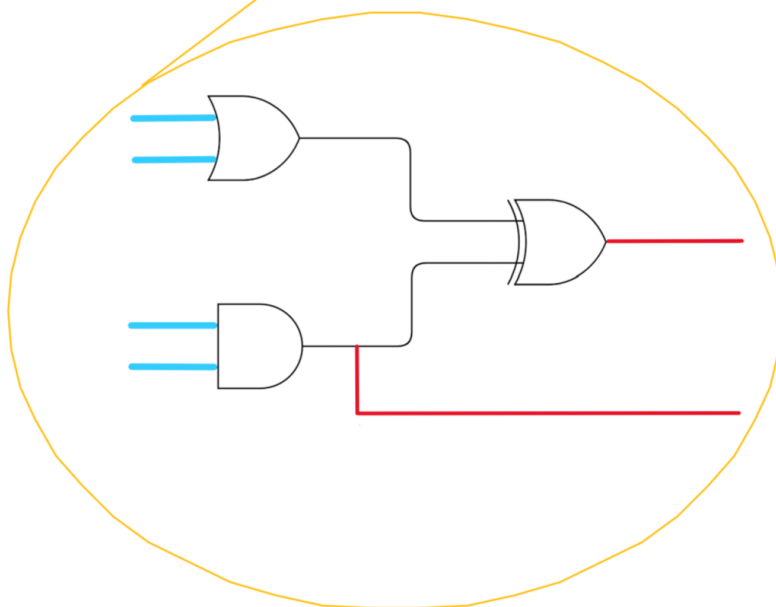
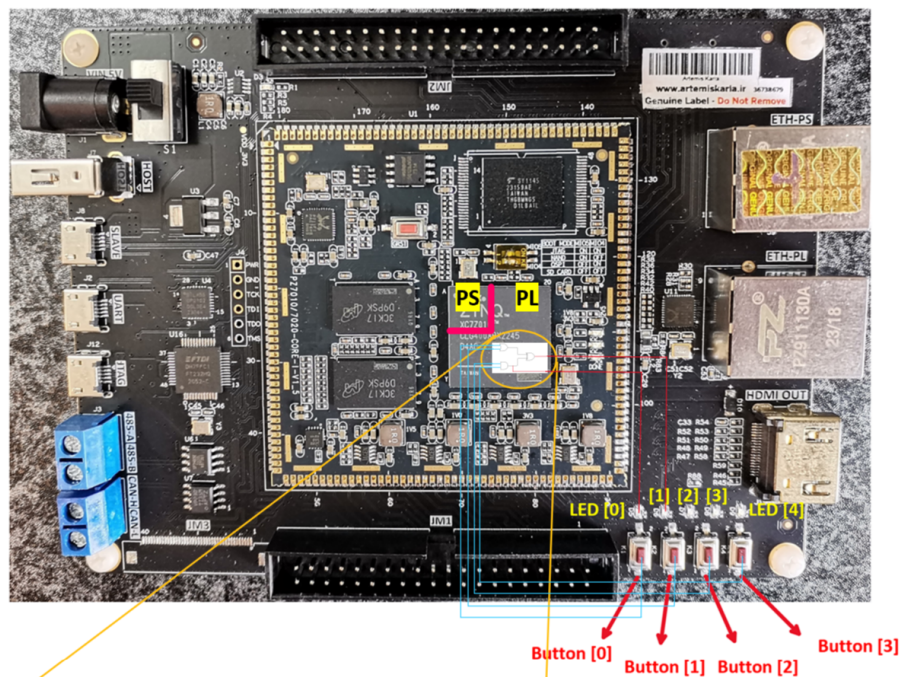
با زدن گزینه device Program و تنظیم درست فایل bitstream پروژه، تراشه را پروگرام کنید:



ه) تست پروژه روی برد:

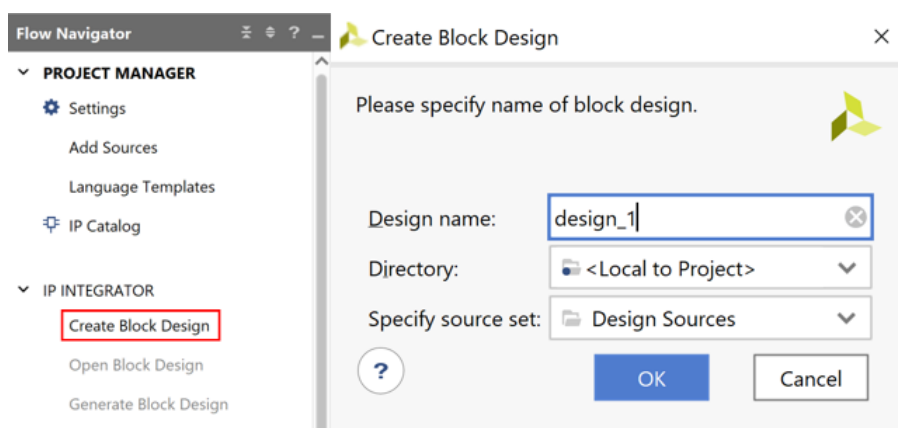


شکل کلی مدار در برد پس از پروگرام کردن FPGA به این شکل خواهد بود: (توجه کنید که LED ها active high و کلیدها active low هستند)

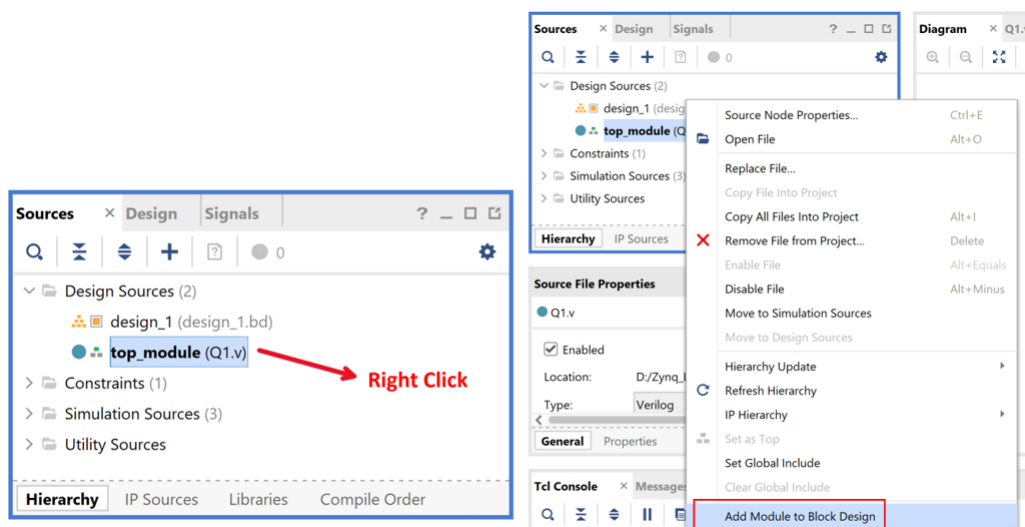


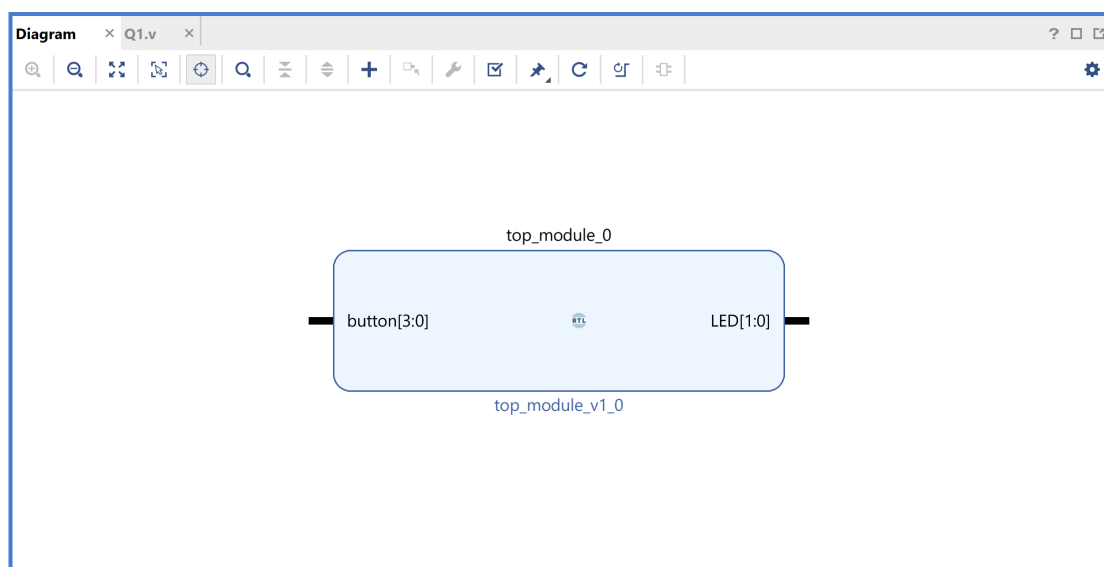
۳-۲-۴ - کار با Block Design در Vivado:

در این مرحله از فضای گرافیکی (Block Design or Diagram) در Vivado استفاده می‌کنید. این محیط گرافیکی را برای راحتی کار شما قرار داده تا بتوانید به راحتی ماژول‌های خود و IP Core های Xilinx را به آن اضافه کرده و به هم متصل کنید و شهودی تصویری و گرافیکی از سخت‌افزار نهایی که در FPGA ساخته می‌شود، داشته باشید. ابتدا یک دیاگرام بسازید:

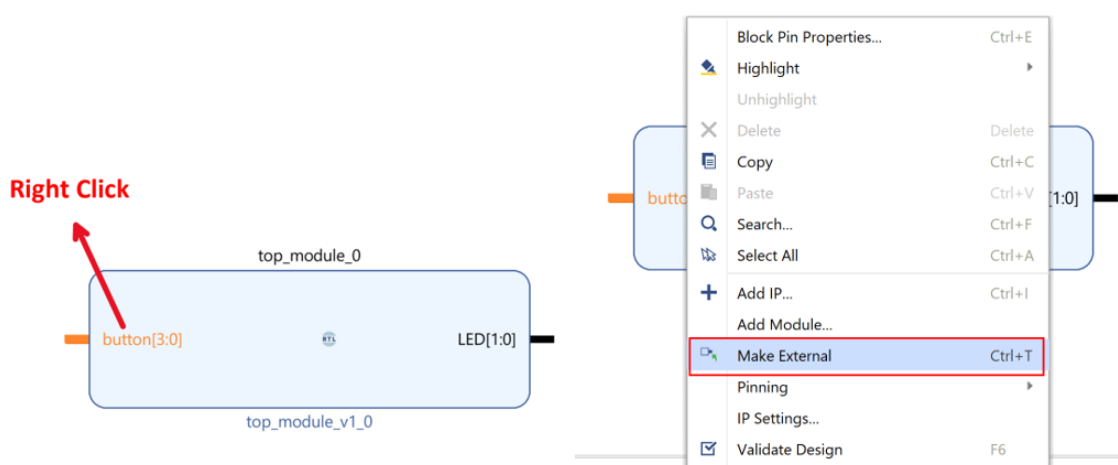


ماژول خود را به محیط گرافیکی Vivado اضافه کنید:

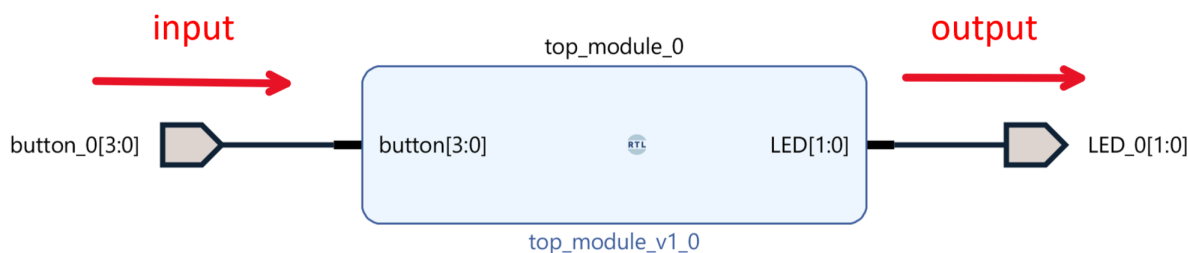




ورودی خروجی‌های ماژول را Make External کنید تا آماده اتصال به پین‌های تراشه باشد:



حال متناسب با نام جدیدی که External Port هر پورت ورودی یا خروجی گرفته، Constraint File خود را آپدیت کنید: (همچنین می‌توانید با کلیک بر روی پورت اکسترنال شده، نام آن را از پنجره Properties External Port به دلخواه تغییر دهید)



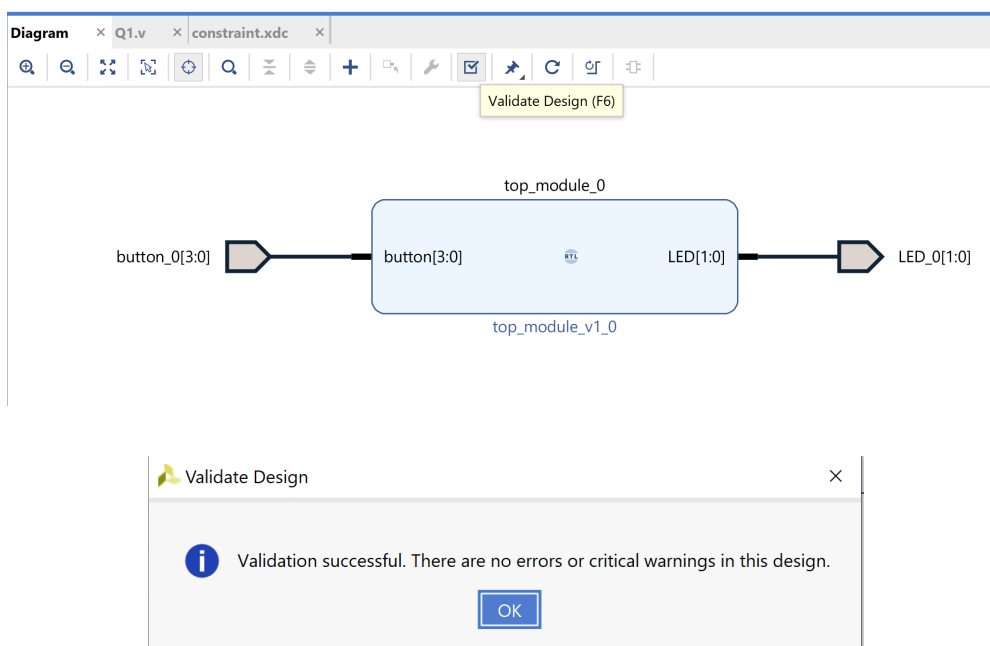
```

Diagram x Q1.v x constraint.xdc x
D:/Zynq_LC_labs/1_zynq/Q1/Q1.srscs/constrs_1/new/constraint.xdc

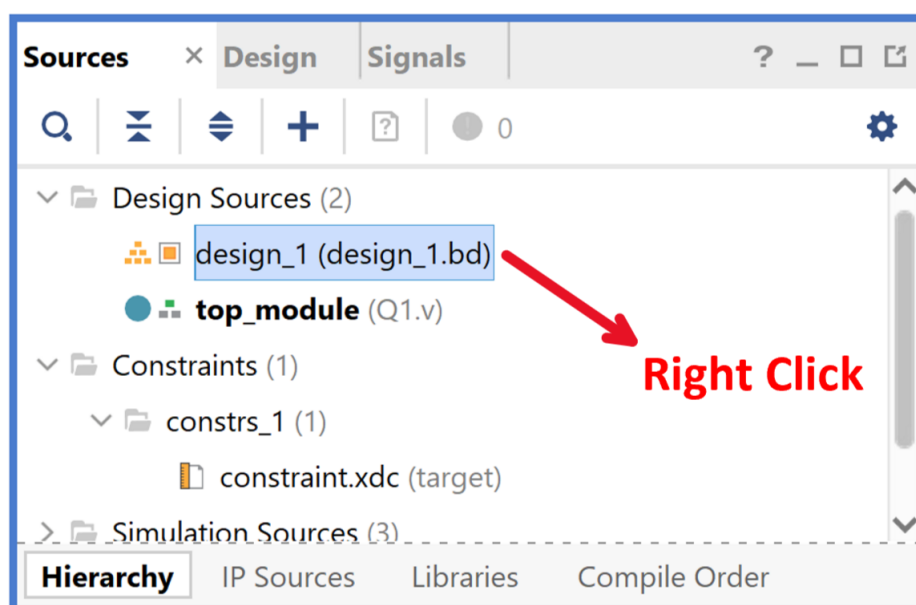
1 set_property PACKAGE_PIN T10 [get_ports {button_0[0]}]
2 set_property PACKAGE_PIN T11 [get_ports {button_0[1]}]
3 set_property PACKAGE_PIN T15 [get_ports {button_0[2]}]
4 set_property PACKAGE_PIN T14 [get_ports {button_0[3]}]
5 set_property PACKAGE_PIN V20 [get_ports {LED_0[0]}]
6 set_property PACKAGE_PIN W19 [get_ports {LED_0[1]}]
7 set_property IOSTANDARD LVCMOS33 [get_ports {button_0[2]}]
8 set_property IOSTANDARD LVCMOS33 [get_ports {button_0[1]}]
9 set_property IOSTANDARD LVCMOS33 [get_ports {button_0[3]}]
10 set_property IOSTANDARD LVCMOS33 [get_ports {button_0[0]}]
11 set_property IOSTANDARD LVCMOS33 [get_ports {LED_0[1]}]
12 set_property IOSTANDARD LVCMOS33 [get_ports {LED_0[0]}]
13
14

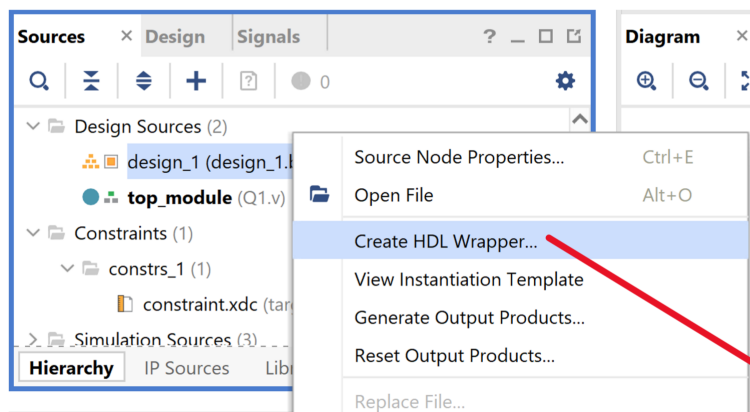
```

با زدن گزینه Design Validate از صحت اولیه دیاگرام گرافیکی خود از نظر نوع اتصالات، تعداد بیت پورت‌های متصل به هم، هماهنگی High Active و Low Active بودن پورت‌های متصل به هم، اتصال صحیح کلاک و ریست مدار و... اطمینان حاصل کنید: (توجه کنید که این گزینه صرفاً برای چک کردن صحت موارد ابتدایی دیاگرام است و به هیچ وجه جای سنتز را نمی‌گیرد)

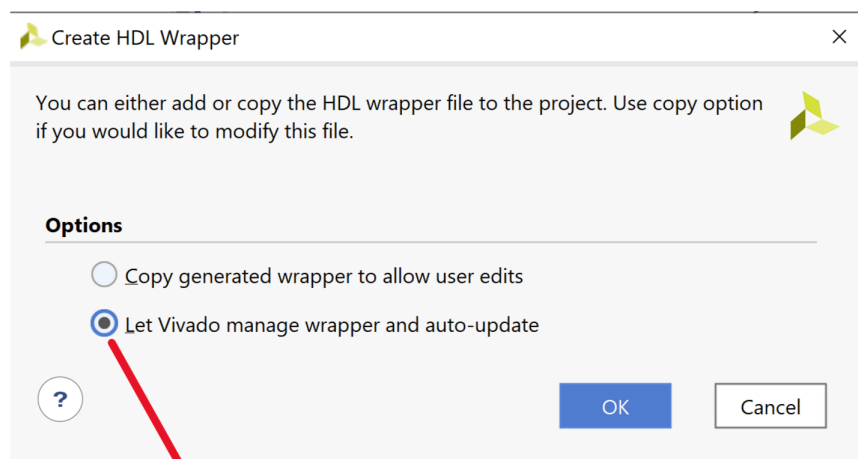


نکته مهمی که در کار با دیاگرام گرافیکی باید بدانید این است که این فضا صرفاً برای راحتی کار شما طراحی شده و tool Synthesis نرم افزار Vivado برای سنتز مدار در نهایت از شما یک کد وریلاگ می‌خواهد؛ پس باید راهی باشد تا سخت افزار طراحی شده در دیاگرام را به یک کد وریلاگ تبدیل و با آن توصیفش کنیم:



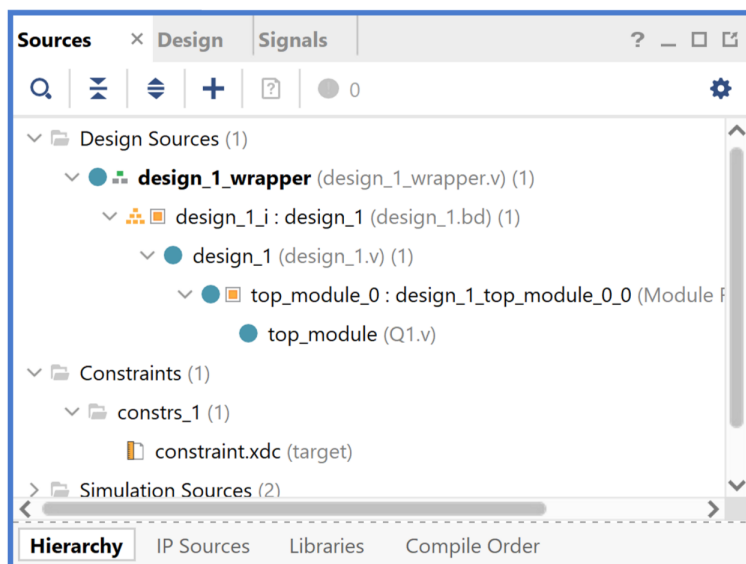


ساختن یک HDL Wrapper به عنوان ماژول وریلاگ توصیف کننده دیاگرام گرافیکی



به Vivado اجازه دهید در صورت تغییر دیاگرام، خودش به صورت خودکار HDL Wrapper توصیف کننده آن را آپدیت کند.

فایل وریلاگ `design_1_wrapper.v` به عنوان `top module` نهایی پروژه



حال لازم است مراحل پیاده‌سازی روی بورد را مجدداً طی کنید و نتیجه را تست کنید.

Synthesis and Implementation Out-of-date [details](#) 

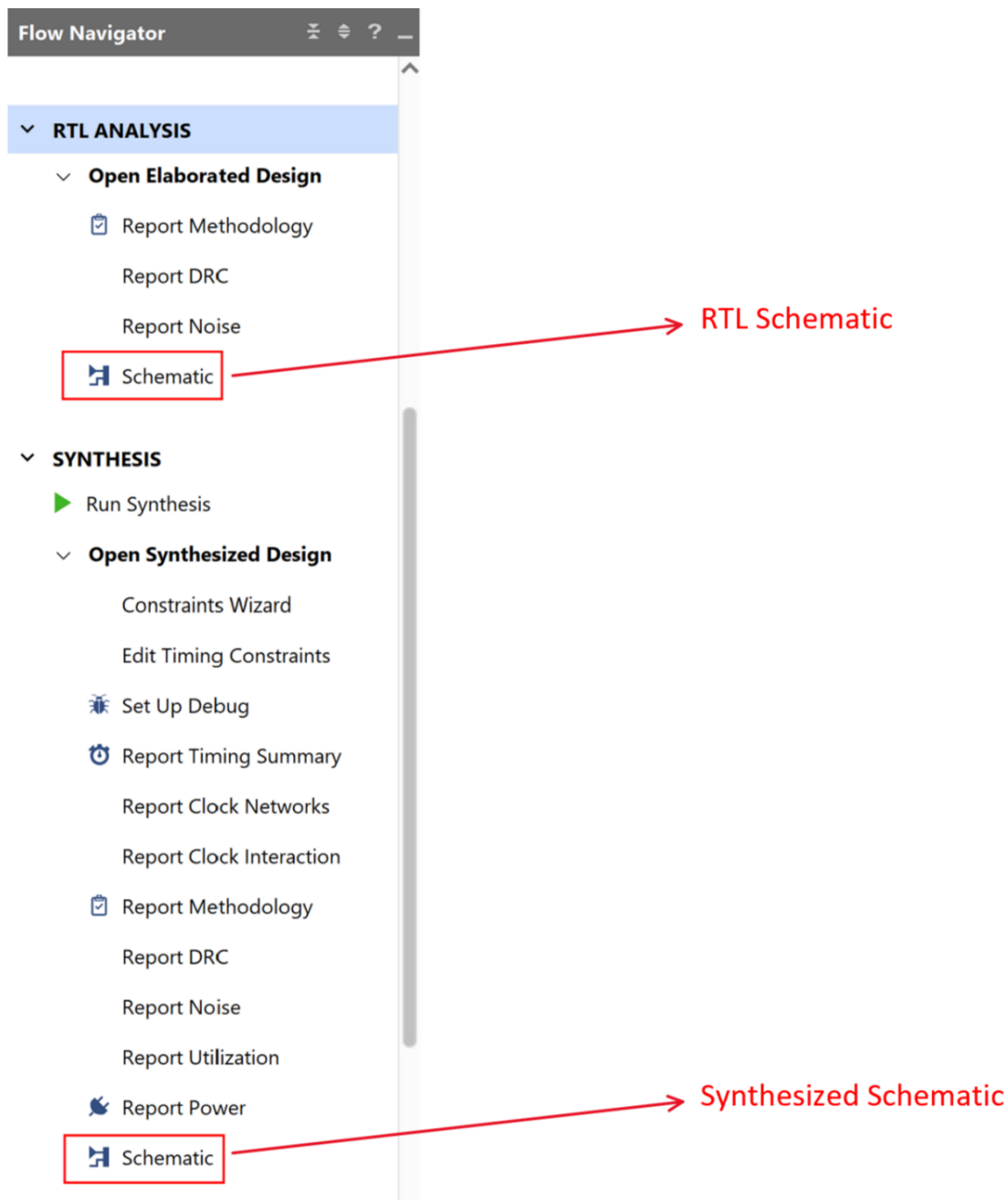
۳-۳- بخش دوم (زمان تقریبی: ۲۵ دقیقه)

کد وریلگی بنویسید که مدار تابع منطقی زیر (که همان تابع بخش ۲-۶ آزمایش اول است) را توصیف کند و سپس آن را در FPGA پیاده‌سازی کنید.

$$f = \Sigma(0, 2, 4, 5, 8, 10, 12, 13)$$

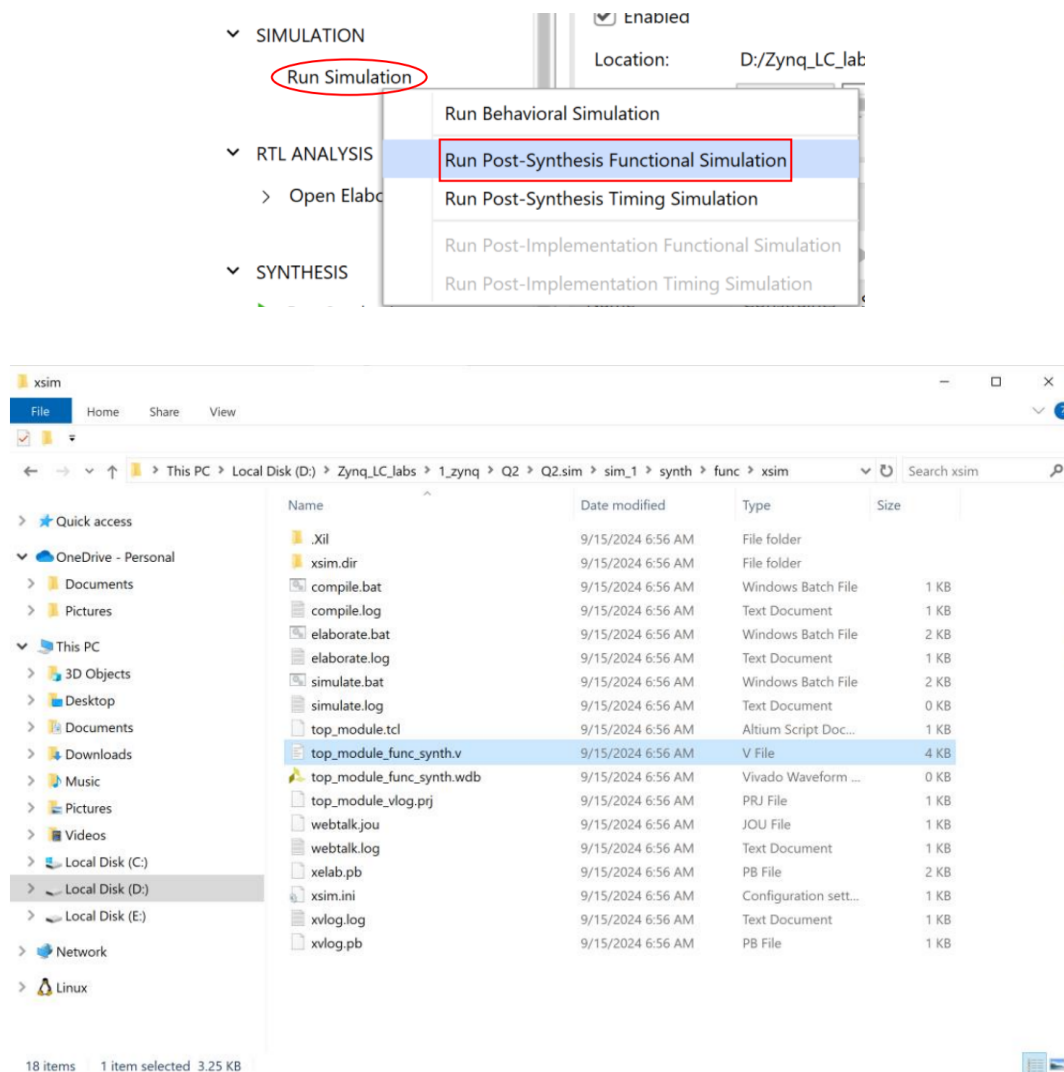
توجه کنید که لازم نیست قبل از توصیف، مدار را بهینه کنید چون این فرایند را نرم‌افزار به صورت خودکار انجام خواهد داد. با استفاده از کلیدهای فشاری به عنوان ورودی و LEDها به عنوان خروجی، مدارهایتان را راستی‌آزمایی نمایید.

۱. شماتیک مدار توصیف شده (RTL Schematic) و پس از سنتز، شماتیک مداری که روی FPGA پیاده شده (Technology Schematic – Synthesized Design Schematic) را مشاهده کنید و نتیجه را در گزارش بنویسید. (RTL Schematic را با طراحی خود در آزمایش اول مقایسه کنید). برای مشاهده RTL Schematic و Technology Schematic از بخش RTL Synthesis و Analysis گزینه Schematic را انتخاب نمایید.



۲. با بررسی فایل وریلاگ پس از سنتز، حاصل بهینه‌سازی لاجیک مدار را مشاهده و ارزیابی کنید و در گزارش بنویسید.

پس از سنتز Run Post-Synthesis Functional Simulation را انتخاب کنید و پس از تولید ماژول Post-Synthesis مطابق تصاویر زیر این فایل را از فولدر پروژه پیدا کنید:

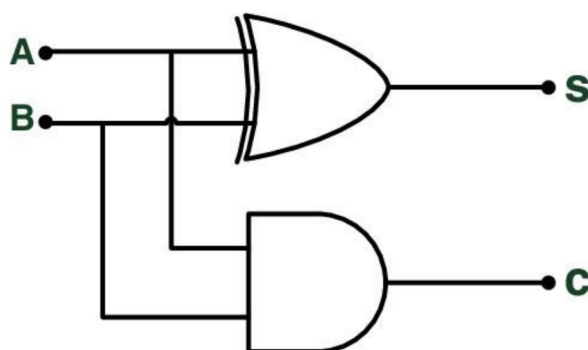


Post-Synthesis Functional Simulation یعنی انجام شبیه‌سازی پس از سنتز. در بخش اول آزمایش شما Behavioral Simulation را انجام دادید که صرفاً بررسی صحت عملکردی کد وریلاگ بود. اما این بار، پس از سنتز که اطلاعات بیشتری از مدار نهایی (که قرار است درون FPGA پیاده شود) بدست آمده، می‌توان با داشتن جزئیات بیشتر از مدار، شبیه‌سازی دقیق‌تری از آن داشت.

۳-۴ - بخش سوم (زمان تقریبی: ۴۰ دقیقه)

هدف از این بخش طراحی یک Full Adder تکبیتی است.

۱. ابتدا کد وریلاگ ماژول یک Half Adder تکبیتی را بنویسید:



۲. سپس یک top module طراحی کنید و با instantiate کردن دو Half Adder تکبیتی درون آن، یک Full Adder تکبیتی بسازید. (در نحوه instantiate کردن و تایپ مناسب برای ورودی خروجی ماژول‌ها در اتصالات میان آنها دقت کنید)

۳. ابتدا پروژه خود را طبق **مراحل simulation بخش اول**، در Vivado شبیه‌سازی کرده و از صحت عملکرد آن اطمینان حاصل کنید.

۴. نتیجه را پس از انجام **مراحل پیاده‌سازی روی برد**، به کمک کلیدها و LED ها تست کنید.

۵. این بار به جای instantiate کردن از Half Adder ها درون یک top module، آن‌ها را به فضای گرافیکی Vivado (Block Design) اضافه کنید و با برقراری اتصالات لازم میان دو Half Adder، یک Full Adder بسازید. نهایتاً با انجام **مراحل مربوط به دیاگرام بلاک دیزاین** و **مراحل پیاده‌سازی روی برد**، نتیجه را به کمک کلیدها و LED ها تست کنید.

۳-۵- بخش چهارم (زمان تقریبی: ۴۰ دقیقه)

هدف از این بخش آشنایی با پیاده‌سازی مدارات با استفاده از multiplexer بر روی FPGA است. در این بخش نیز فقط مجاز به استفاده از عبارات Continuous Assignment به عنوان مثال، برای پیاده‌سازی با multiplexer باید از عبارت زیر استفاده کنید:

```
wire x;  
assign x = ... ? ... : ...;
```

در مدار این بخش، پس از پیاده‌سازی، شماتیک مدار خود را (RTL Schematic) در Vivado به ترتیبی که در بخش اول همین آزمایش توضیح داده شد، مشاهده کنید. همچنین برای تست مدار این بخش بر روی FPGA، از کلیدها و LED های برد استفاده کنید. جبر بول زیر را تنها با استفاده از multiplexer بر روی FPGA پیاده‌سازی کنید.

$$F = CAB + \overline{C} \overline{(A + B)}$$

۳-۶- بخش پنجم (زمان تقریبی: ۴۰ دقیقه)

در این بخش یک Priority Encoder طراحی کنید. مدار شما چهار ورودی دارد با شماره‌های فرضی ۱ تا ۴ که همان چهار کلید روی برد هستند و در خروجی، بزرگترین شماره کلید فشرده شده را روی LEDها نمایش می‌دهد. (طبیعتاً به سه LED نیاز دارید زیرا بیشترین شماره $3'b100 = 4$ است) برای مثال اگر کلیدهای ۱ و ۳ همزمان فشرده شده باشند، بر روی LEDها باید عدد ۳ که بزرگتر است نمایش داده شود یعنی $3 = 3'b011$

۴- گزارش

در بخشی از قسمت های آزمایش بیان شد که نتیجه را در گزارش قید کنید. ابتدا موارد مربوطه را ذکر کرده و سپس در خصوص آنچه از این آزمایش آموختید گزارش ۱ الی ۲ صفحه‌ای (مطابق **تمپلیت** ارجاع شده در ابتدای این داکيومنت) بنویسید. دیدگاه خود نسبت به بخش‌های مختلف آزمایش را به انضمام پیشنهادهاitan بنویسید.

اطلاعات تکمیلی آزمایش سوم

شما تاکنون با مدارهای دیجیتال کار کردید و در آزمایش‌های گذشته، مدارهای شامل چند گیت منطقی را با استفاده از گیت‌های منفصله‌ی خانواده 74xx، با برقراری اتصالات لازم بر روی یک بردبرد، پیاده‌سازی نمودید و عملکرد آنها را بررسی کردید. در آن آزمایش‌ها، بردبرد بکار گرفته شده و اتصالات آنها همگی بصورت فیزیکی بودند. اما تصور کنید مدار دیجیتالی که قصد طراحی آن را دارید تعداد بسیار زیادتری گیت منطقی دارد، در واقع برای رسیدن به عملکرد مدنظرتان لازم است تعداد زیادی گیت داشته باشد (هزاران هزار گیت). برای مثال پردازنده‌ای که درون تلفن همراه شماست، یک مدار دیجیتال و منطقی است و برای اینکه بتواند تمام محاسبات و عملیات‌های مدنظر را انجام دهد لازم است ده‌ها هزار گیت درونش باشد (و طبعاً تعداد بسیار بیشتری ترانزیستور که گیت‌های منطقی از آنها ساخته شده). همانطور که متوجه شده اید برای طراحی چنین مداری دیگر نمی‌توان آن را در سطح گیت Gate Level در پروتئوس طراحی کرد و بر روی برد برد بست. پس راه‌حل چیست؟!

HDL (Hardware Description Language) یا زبان‌های توصیف سخت‌افزار، در واقع ما به این وسیله و با این ادبیات مدار مدنظرمان را توصیف می‌کنیم. توجه کنید که در زبان‌های HDL درست است که کد می‌زنیم اما قرار نیست کد ما بر روی چیزی مثل پردازنده لپ‌تاپ اجرا شود (مانند درس برنامه نویسی ترم اول) بلکه HDL راهی است برای توصیف مدار دیجیتالی که در ذهنمان است (تصور کنید یک متن می‌نویسید تا وضعیت یک اتاق را توصیف کنید، در آن متن قرار نیست دستوری به کسی بدهید تا اجرا کند، بلکه صرفاً در حال توصیف هستید پس به طور کلی داشتن ترتیب بین خطوط کد HDL بی‌معنی است، قرار نیست دستوراتی پشت سر هم و با ترتیب اجرا شوند، مانند کدها و زبان‌های نرم‌افزاری).

یکی از HDLها که در دروس دانشگاه می‌آموزید Verilog است. این زبان اولین بار در سال ۱۹۸۴ ابداع شد و چند سال بعد به استاندارد IEEE درآمد. (در این متن برای سادگی از این پس Verilog را به فارسی خواهیم نوشت یعنی وریلاگ) در این مرحله باید انتظار داشته باشید ابزاری وجود داشته باشد تا کد وریلاگ شما را به مدار دیجیتال مذکور تبدیل کند؛ به این ابزار اصطلاحاً Synthesis Tool گفته می‌شود یعنی ابزار سنتز مدار. بعد از سنتز شدن مدار و طی کردن یک فرایند و flow مشخص در نرم‌افزارهای مخصوص این کار، به layout مدار مجتمع مدنظر خواهیم رسید که آماده ساخته شدن (اصطلاحاً fabricate شدن) در شرکت‌هایی مثل TSMC است که کارشان تولید IC است.

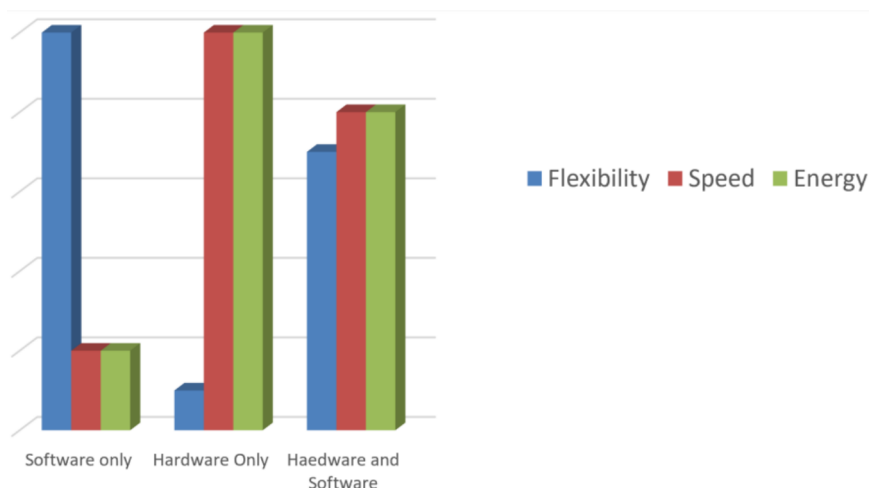
به این کار طراحی ASIC (Application Specific Integrated Circuit) می‌گویند؛ یعنی طراحی یک IC دیجیتال خاص که دقیقاً و تنها همان عملکرد و کاربردی را دارد که ما در کد وریلاگ توصیفش کردیم. اما هدف ما در این درس طراحی ASIC نیست! پس قرار است با وریلاگ چه کنیم؟! فقط در حد شبیه‌سازی کردن مدار توصیف شده با وریلاگ و بررسی صحت عملکرد آن؟! خیر، گرچه این کار را هم مختصراً انجام خواهیم داد.

FPGA (Field Programmable Gate Array) سخت‌افزاری است که مدار ثابت با عملکرد مشخصی درون آن شکل نگرفته، بلکه طوری طراحی شده تا بتواند مدارات دیجیتال دلخواه ما را درون خود بسازد. دنیای FPGA بسیار وسیع بوده و نحوه‌ی کارکرد آن نسبتاً پیچیده و از موضوع درس و این آزمایش و دستور کار خارج است. برای کاربرد جاری و تسهیل موضوع و با یک ساده‌سازی نسبتاً عمیق، FPAG را می‌توان یک بردبرد الکترونیکی دانست. _ این بدان معناست که در یک FPGA شما می‌توانید چند صد گیت را قرار داده و با کمترین زحمت، اتصالات میان آنها را برقرار کنید. در ساده‌ترین حالت تصور کنید یکسری گیت منطقی در بلوک‌هایی مشخص درون FPGA وجود دارد و بین آنها سیم‌ها و سوئیچ‌هایی قرار گرفته است؛ حال این سیم‌ها طوری گیت‌ها را به هم متصل می‌کنند تا مداری که ما (با وریلاگ) توصیف کرده‌ایم درون بخشی از FPGA ساخته شود؛ درواقع خانواده‌های FPGA (همانطور که از اسمشان مشخص است) سخت‌افزاری قابل برنامه‌ریزی (programmable) هستند. طبیعتاً انتظار دارید FPGAها هم Synthesis tool داشته باشند تا کد وریلاگ ما را به مدار دیجیتال و سپس به ادبیاتی درآورد که پروگرامر FPGA آن را بفهمد و درون FPGA بسازد.

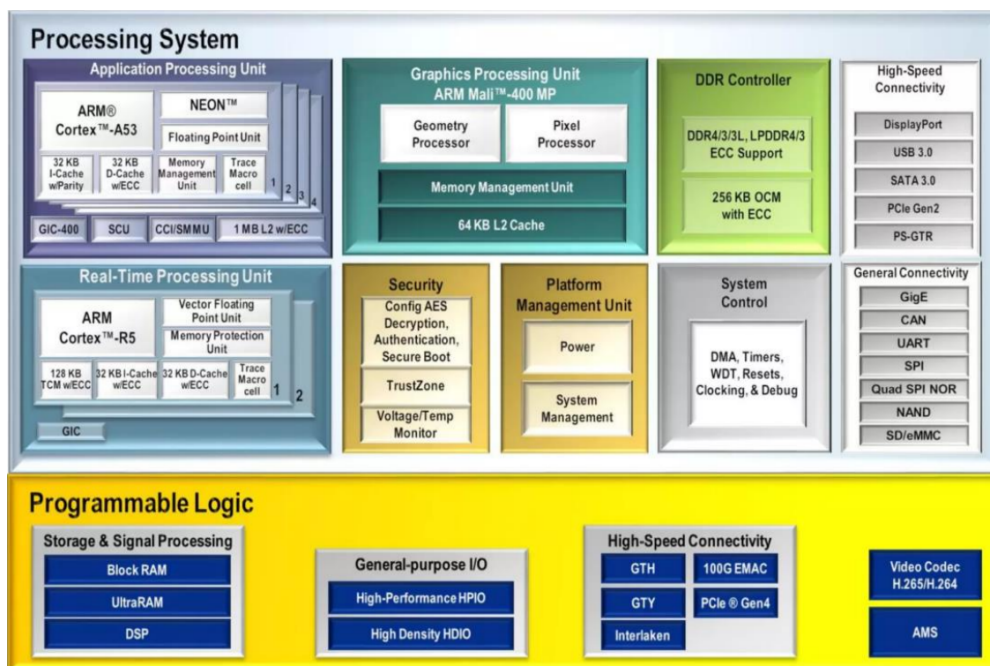
Xilinx یکی از شرکت‌هایی است که FPGA می‌سازد و نرم‌افزار Vivado را به عنوان ابزاری برای این فرایند در جدیدترین تراشه‌های خود (از نسل ۷ به بعد) معرفی کرده است. شما برای آزمایش‌های این درس لازم است **نرم‌افزار Vivado (نسخه ۲۰۱۹.۱)** را نصب نمایید. اما کاربرد FPGA چیست؟ دو نوع الگوریتم داریم، یکسری الگوریتم‌ها ثابت یا static هستند یعنی عملیات مشخصی روی داده‌های ورودی انجام می‌شود (مثلاً وارون یک ماتریس ورودی باید محاسبه شود). عمده الگوریتم‌های مربوط به پردازش سیگنال و پردازش تصویر از این دسته اند. از طرفی این پردازش‌ها باید با بیشترین سرعت و کمترین تأخیر انجام شود و به جهت static بودن الگوریتم از نظر منابع سخت‌افزاری مصرفی امکان پیاده‌سازی مداری که خاص انجام این الگوریتم باشد، وجود دارد؛ بنابراین مدار آنها را در FPGA پیاده می‌کنیم.

دسته دوم الگوریتم‌های adaptive هستند یعنی الگوریتم‌هایی که بر از شرایط گوناگون و نامشخص اند (به زبان ساده پر از if و else اند). به جهت منابع و محدودیت‌های دیگر امکان پیاده کردن مدار خاص این الگوریتم‌ها وجود ندارد. لذا برای اجرای آنها از پردازنده‌ها و میکروکنترلرها استفاده می‌کنیم.

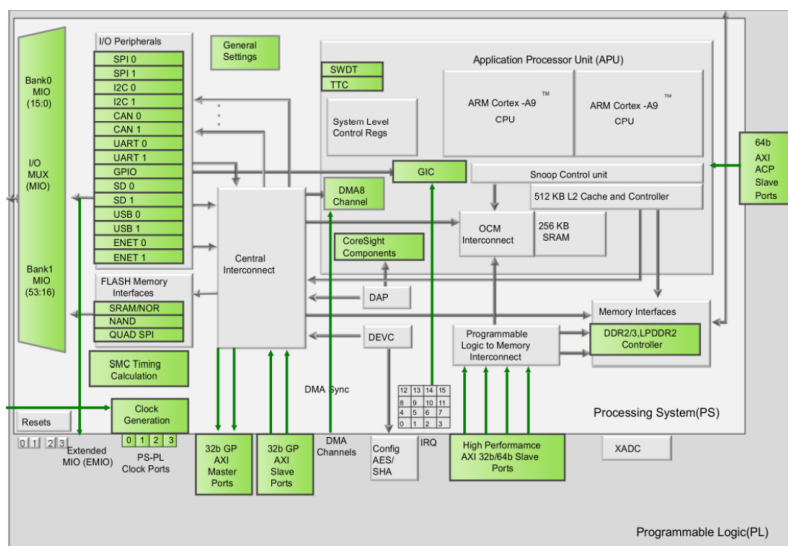
پروژه‌های ما دارای بخش‌هایی با الگوریتم‌های static و بخش‌هایی با الگوریتم‌های adaptive اند. برای برآورده کردن این نیاز شرکت Xilinx در سال ۲۰۱۱ تراشه‌ای را معرفی کرد که یک SoC (System on Chip) است یعنی یک سیستم داخل یک تراشه جا گرفته است. این IC به طور کلی شامل یک بخش FPGA است که به آن PL (Programmable Logic) می‌گوییم و یک بخش پردازنده که به PS (Processing System) معروف است. PS علاوه بر پردازنده دو هسته‌ای ARM که دارد، شامل موارد گوناگون دیگری همچون حافظه، پورت‌ها، پروتکل‌ها و پریفرال‌های پیاده شده می‌باشد.



شکل ۳-۳: مقایسه سیستم‌های دارای بخش‌های سخت‌افزاری (FPGA) و نرم‌افزاری (میکرو) و تلفیق آنها از نظر انعطاف، سرعت و توان مصرفی



شکل ۳-۴: بخش‌های مختلف درونی تراشه Zynq UltraScale+



شکل ۳-۵: نمایی از درون تراشه Zynq 7000

برای کسب اطلاعات بیشتر در مورد تراشه Zynq می‌توانید به [Zynq 7000 reference manual](#) مراجعه کنید.



Device Name	Z-7007S	Z-7012S	Z-7014S	Z-7010	Z-7015	Z-7020	Z-7030	Z-7035	Z-7045	Z-7100
Part Number	XC7Z007S	XC7Z012S	XC7Z014S	XC7Z010	XC7Z015	XC7Z020	XC7Z030	XC7Z035	XC7Z045	XC7Z100
Xilinx 7 Series Programmable Logic Equivalent	Artix®-7 FPGA	Artix-7 FPGA	Artix-7 FPGA	Artix-7 FPGA	Artix-7 FPGA	Artix-7 FPGA	Kintex®-7 FPGA	Kintex-7 FPGA	Kintex-7 FPGA	Kintex-7 FPGA
Programmable Logic Cells	23K	55K	65K	28K	74K	85K	125K	275K	350K	444K
Look-Up Tables (LUTs)	14,400	34,400	40,600	17,600	46,200	53,200	78,600	171,900	218,600	277,400
Flip-Flops	28,800	68,800	81,200	35,200	92,400	106,400	157,200	343,800	437,200	554,800
Block RAM (# 36 Kb Blocks)	1.8 Mb (50)	2.5 Mb (72)	3.8 Mb (107)	2.1 Mb (60)	3.3 Mb (95)	4.9 Mb (140)	9.3 Mb (265)	17.6 Mb (500)	19.2 Mb (545)	26.5 Mb (755)
DSP Slices (18x25 MACCs)	66	120	170	80	160	220	400	900	900	2,020
Peak DSP Performance (Symmetric FIR)	73 GMACs	131 GMACs	187 GMACs	100 GMACs	200 GMACs	276 GMACs	593 GMACs	1,334 GMACs	1,334 GMACs	2,622 GMACs
PCI Express (Root Complex or Endpoint) ⁽³⁾		Gen2 x4			Gen2 x4		Gen2 x4	Gen2 x8	Gen2 x8	Gen2 x8
Analog Mixed Signal (AMS) / XADC	2x 12 bit, MSPS ADCs with up to 17 Differential Inputs									
Security ⁽²⁾	AES and SHA 256b for Boot Code and Programmable Logic Configuration, Decryption, and Authentication									

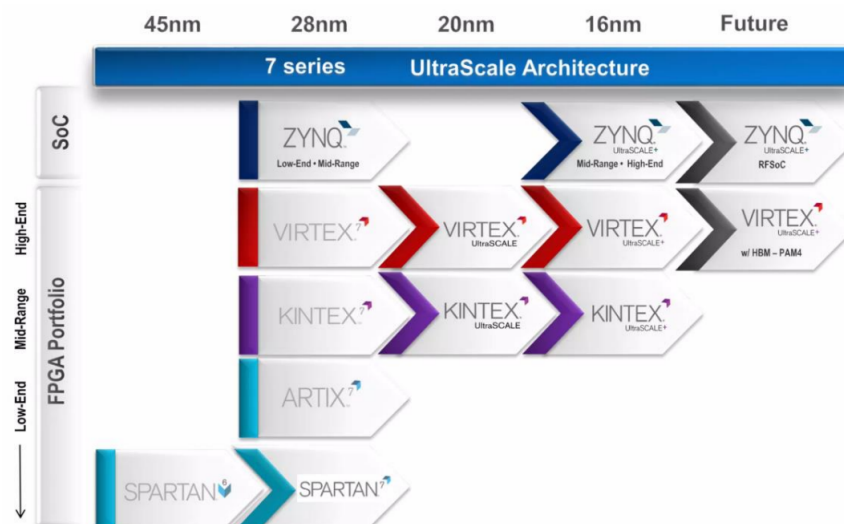
	Device Name	Z-7007S	Z-7012S	Z-7014S	Z-7010	Z-7015	Z-7020	Z-7030	Z-7035	Z-7045	Z-7100
	Part Number	XC7Z007S	XC7Z012S	XC7Z014S	XC7Z010	XC7Z015	XC7Z020	XC7Z030	XC7Z035	XC7Z045	XC7Z100
Processing System	Processor Core	Single-core ARM Cortex-A9 MPCore™ with CoreSight™			Dual-core ARM Cortex-A9 MPCore™ with CoreSight™						
	Processor Extensions	NEON™ & Single / Double Precision Floating Point for each processor									
	Maximum Frequency	667 MHz (-1); 766 MHz (-2)			667 MHz (-1); 766 MHz (-2); 866 MHz (-3)			667 MHz (-1); 800 MHz (-2); 1 GHz (-3)			667 MHz (-1) 800 MHz (-2)
	L1 Cache	32 KB Instruction, 32 KB data per processor									
	L2 Cache	512 KB									
	On-Chip Memory	256 KB									
	External Memory Support ⁽¹⁾	DDR3, DDR3L, DDR2, LPDDR2									
	External Static Memory Support ⁽¹⁾	2x Quad-SPI, NAND, NOR									
	DMA Channels	8 (4 dedicated to Programmable Logic)									
	Peripherals ⁽¹⁾	2x UART, 2x CAN 2.0B, 2x I2C, 2x SPI, 4x 32b GPIO									
	Peripherals w/ built-in DMA ⁽¹⁾	2x USB 2.0 (OTG), 2x Tri-mode Gigabit Ethernet, 2x SD/SDIO									
Security ⁽²⁾	RSA Authentication, and AES and SHA 256-bit Decryption and Authentication for Secure Boot										
Processing System to Programmable Logic Interface Ports (Primary Interfaces & Interrupts Only)		2x AXI 32b Master 2x AXI 32-bit Slave									
		4x AXI 64-bit/32-bit Memory									
		AXI 64-bit ACP									
		16 Interrupts									

شکل ۳-۶: بخشی از ug585 - امکانات و مشخصات بخش PL و PS مدل های مختلف Zynq

با توجه به قابلیت‌های ویژه Zynq، این تراشه جای خود را در صنعت ایران و جهان باز کرده؛ به طوری که به دلیل تیراژ بالای تولید آن، قیمت آن به صرفه‌تر از تراشه‌هایی بعضاً ضعیف‌تر از آن شده است. به طور مثال در جدول زیر امکانات و قیمت Zynq با سری Artix از FPGA های Xilinx مقایسه شده که نتیجه بسیار جالب است: (بخش FPGA خود این Zynq از نوع Artix است)

	Artix	Zynq	Artix
	XC7A75T	XC7z020	XC10075T
Logic Cells	75k	85k	101k
Block RAM	210	280	270
DSP48	180	220	240
GTx	8	0	8
Price	116\$	104\$	137\$

جدول ۳-۲: مقایسه امکانات و قیمت تراشه Zynq و Artix



شکل ۳-۷: تکنولوژی ساخت Zynq در مقایسه با دیگر تراشه های Xilinx

برای مشاهده تصاویر مفهومی بیشتر در مورد تراشه Zynq می‌توانید به این [لینک](#) مراجعه کنید. برای درک بهتر این مفاهیم اکیداً توصیه می‌شود این [ویدیو](#) را مشاهده بفرمایید.